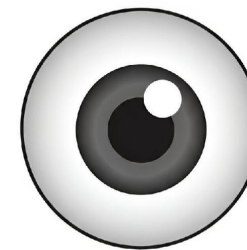
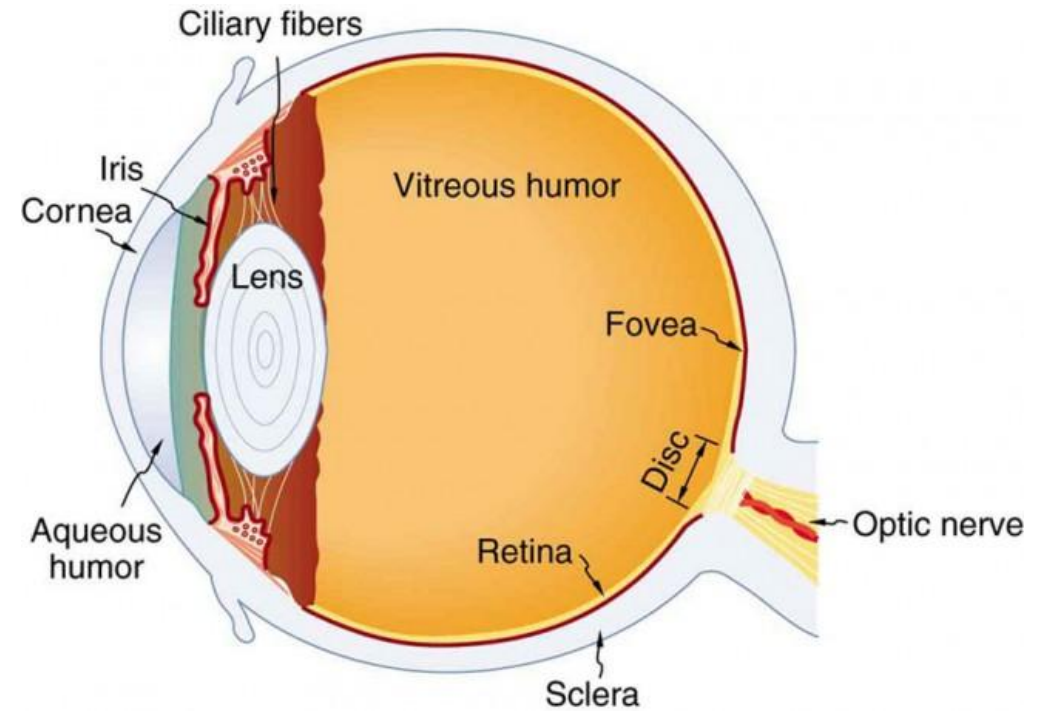


Image formation
In the eyes

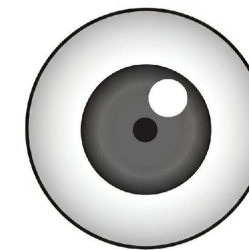
The light reflected from any object enters the eye in the sequence, cornea → pupil → lens. These organelles have different functions. For example, the pupil regulates the amount of light intake. So, in high light settings, your pupils will retract to allow less light in (minimizing any form of damage). and in low light settings they will dilate to let more light in.

The lens focuses the light beam and guides them to the retina. Any object that you focus on is projected on to the fovea. The fovea is responsible for creating the sharpest, most detailed representation of the object viewed by the eye. Other parts of retina also aid in visualizing, but they create less detailed views.

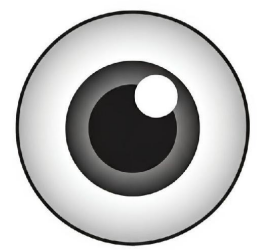
Simply put, imagine focusing on a certain object, the focused object is read in the fovea, and the blurred background in your peripheral sight is created in other parts of the retina.



Normal Light
Regular



Bright Light / Accommodation
Pupil Constriction/Contraction



Dim Light
Dilated

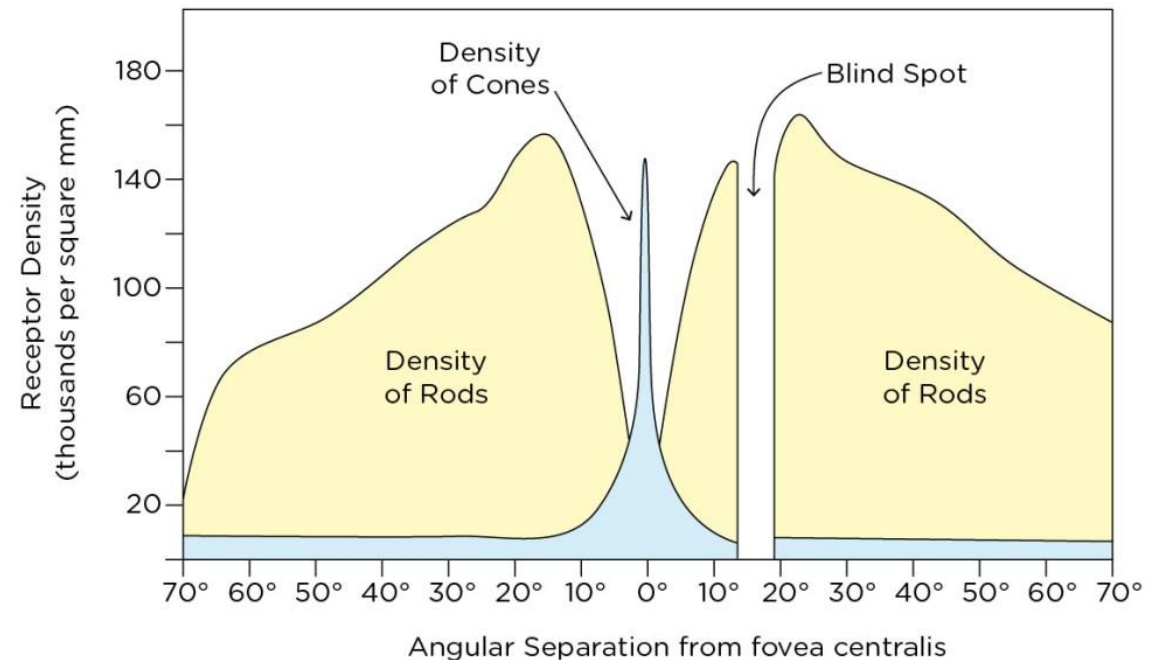
The retina contains the cells that reads visual data. There are two cells responsible for this reading process –

1. Rods
2. Cones

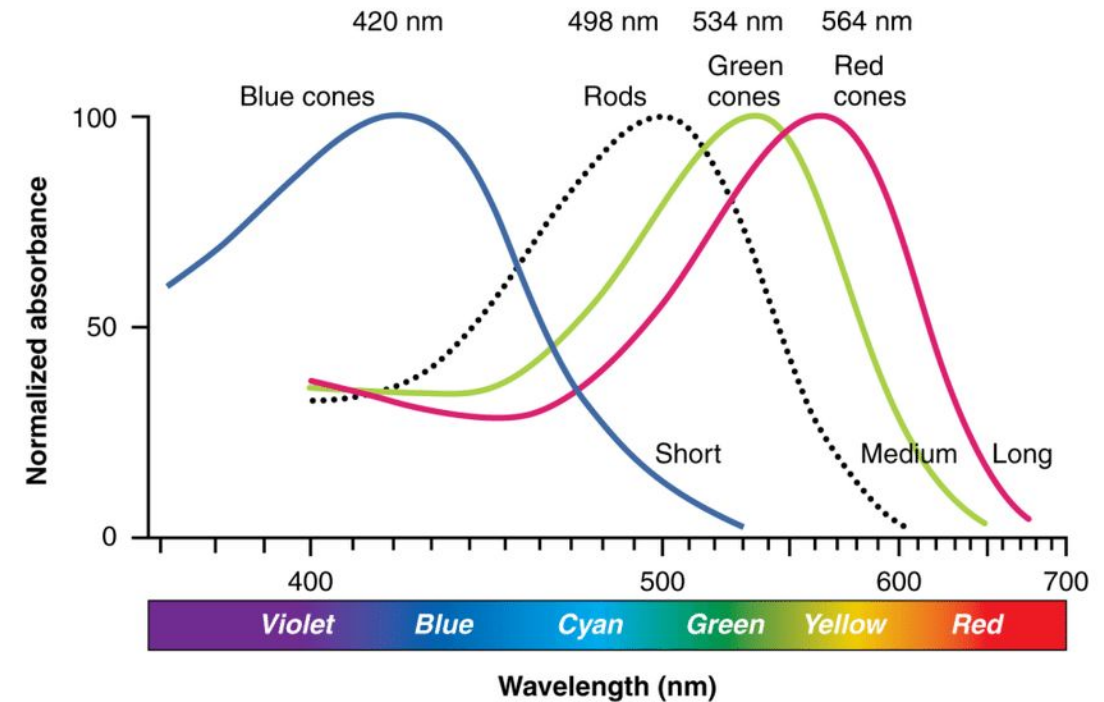
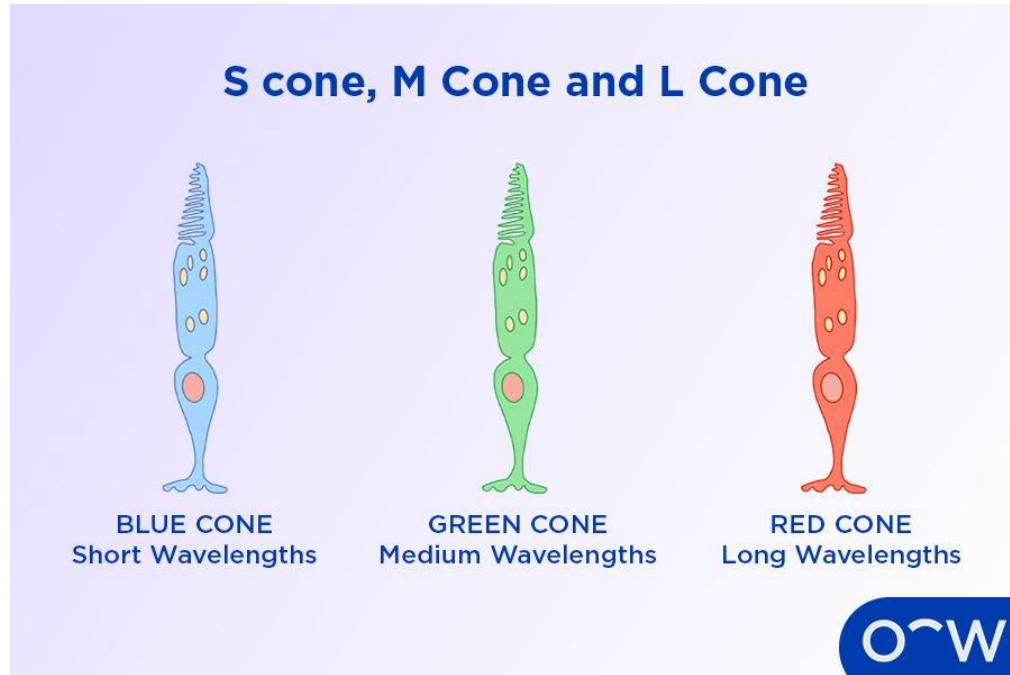
The fovea contains mostly of cones and the rod concentration increases as we move away from the fovea. This indicates that cones are responsible for creating higher resolution views compared to rods. In the figure the blind spot takes place right at the top of the optic nerve.

Cones support photopic vision and rods support scotopic vision. This is just a fancy way of saying, cones work when there is ample light present and rods work when light is low. This also implies that cones are less sensitive to brightness while rods are more sensitive (since they help you see in low light). Additionally, cones can see colors while rods can only sense brightness.

When you walk around in the dark at night in your house, can you see colors very well?



Cones can further be classified into three types – S (sensitive to blue), M (sensitive to green), L (sensitive to red). S stands short, M stands for medium and L stands for long. They refer to the category of wavelength they are sensitive to.



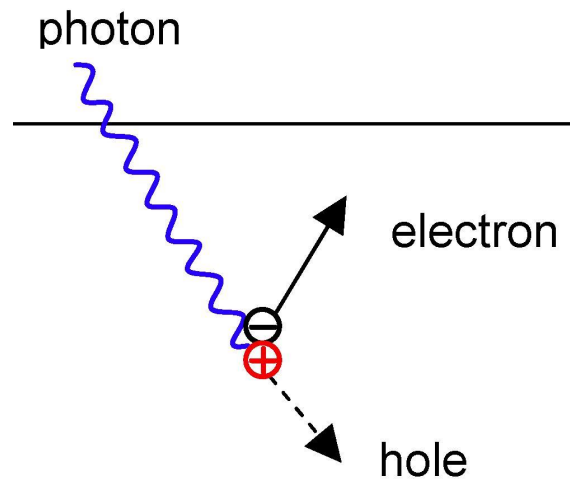
Together, they allow us to see colors. In people with color blindness, some cones may be weakly developed or completely absent.

Image formation

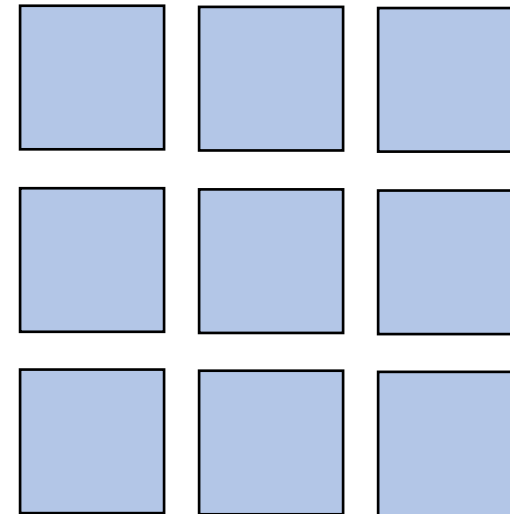
In the camera

We need something similar to our photosensitive cells to create a camera. While we have come a long way, tested a long list of things that can form images, we will not go into them. Lets instead consider a simple toy case as to how we can form images in camera.

Silicon gives us the foundation of converting light energy to electron through photoelectric effect. When a photon hits a silicon atom, it can displace electrons. This is the exact phenomenon that goes into building solar panels. We use a similar principle to make our silicon sensors. Lets skip all the complexities and consider, moving forward, we will represent our silicon sensors as abstract squares -



Photoelectric effect

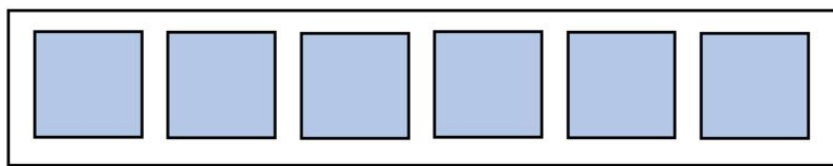
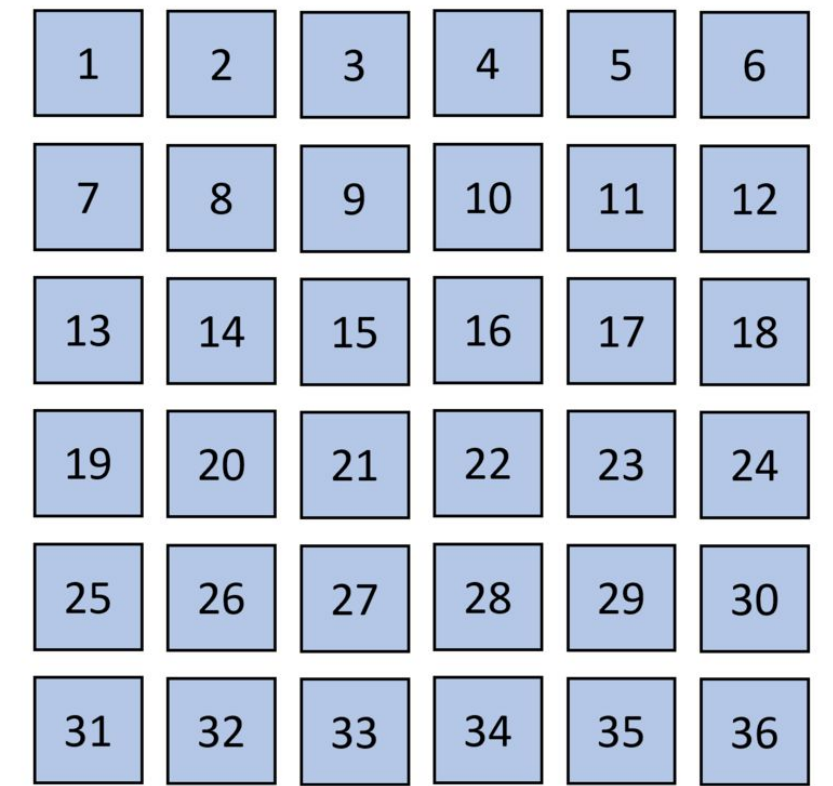


Each square is a sensor

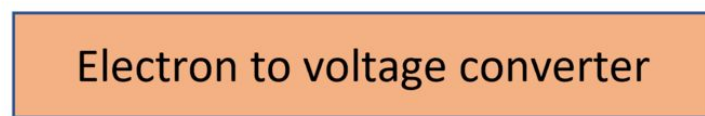
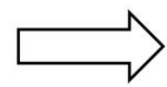
Sensors do not have to be squares, but we will represent them as such in this slide. Do not that each sensor account for one individual pixel of an image.

Each of the sensors in a camera emit electrons upon exposure to light. The number of electrons released depends on how bright of a spot the specific sensor is trying to capture and how long the sensor is exposed to light (generally called exposure time or exposure in short for cameras). The actual engineering challenge in developing proper cameras was not building these sensors, but how to convert these electrons (count them) to get readable electrical signals. There are multiple ways this has been solved, but the two most commonly used technologies are CCD (charge-coupled device) and CMOS (complimentary metal-oxide semiconductor) technology. They both use photosensitive sensors and only differ in how they are converted to usable electrical energy.

Sensor signal conversion (Optional)
CCD

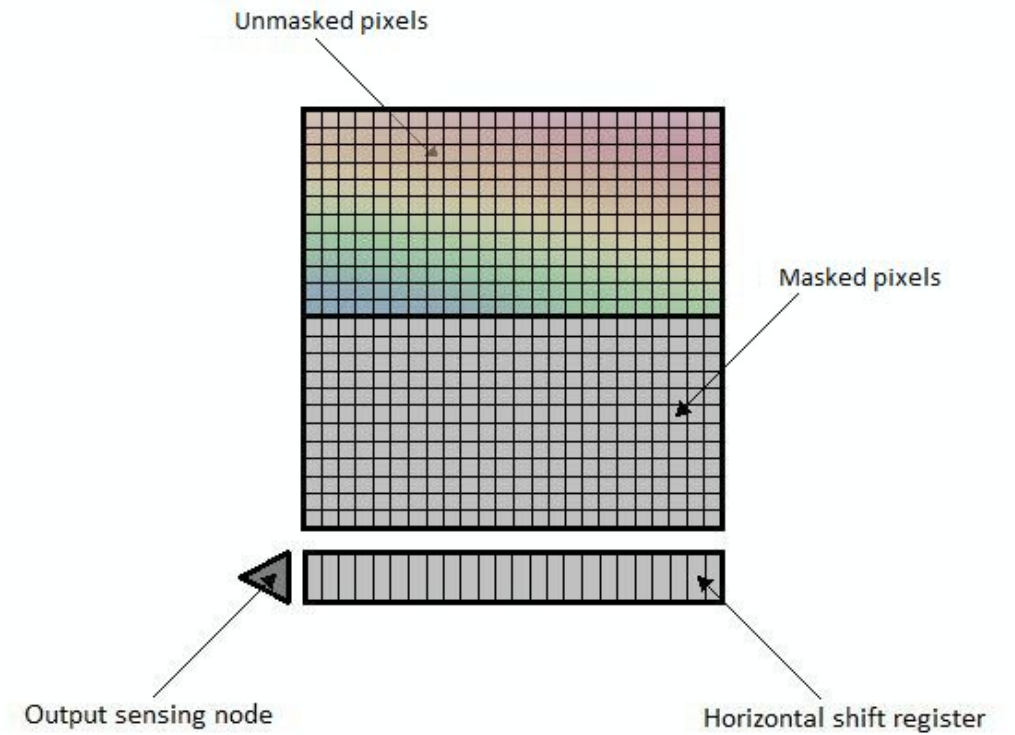
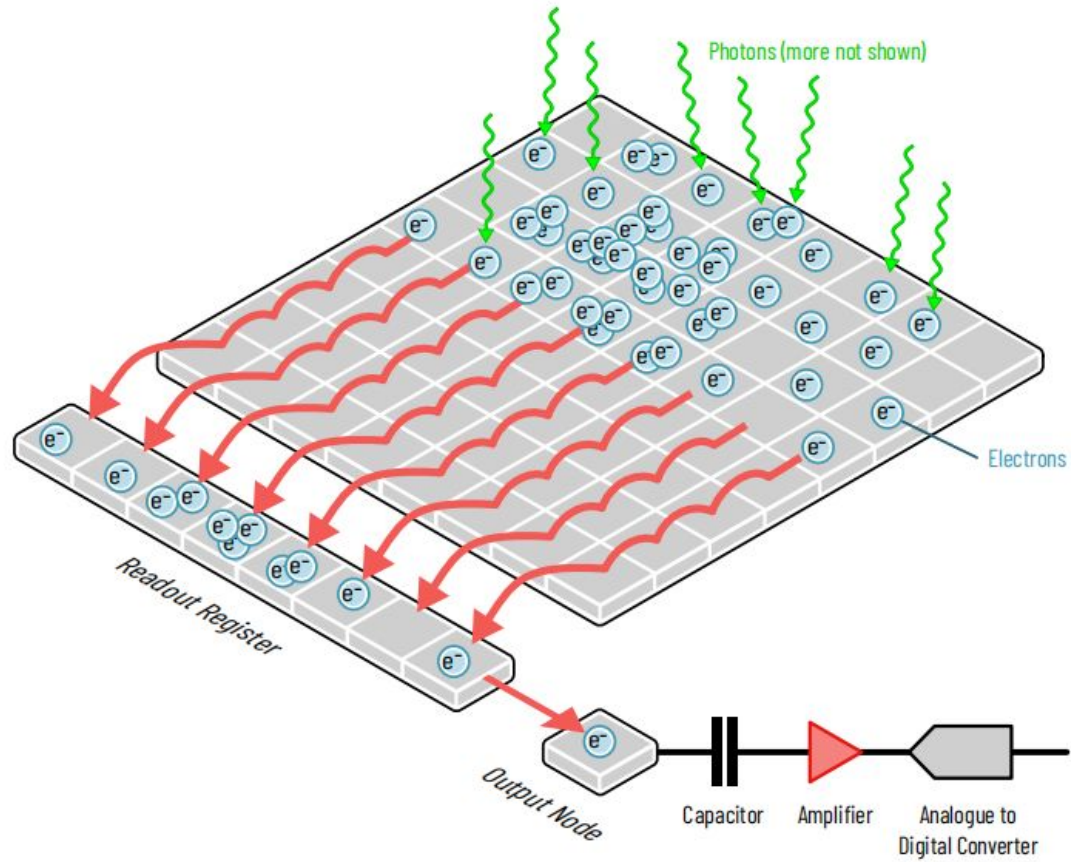


Serial read out register



The numbers shown on the sensors are just indices to keep track of the data movement. The sensors in reality have bins to hold electrons that are transferred through the processor pipeline sequentially. The number of electrons a bin holds depend on how long and how bright the light exposure has been. Only after the electrons passes through the converter and the amplifier does the data become readable voltages that can be quantified through computational processors.

Since the data is read in an opposite manner, they need to be mapped back to original fashion. This is done automatically by the camera processors. Also, even though CCD seems slow, the single processing block for all the sensors, make it quite robust from noises.



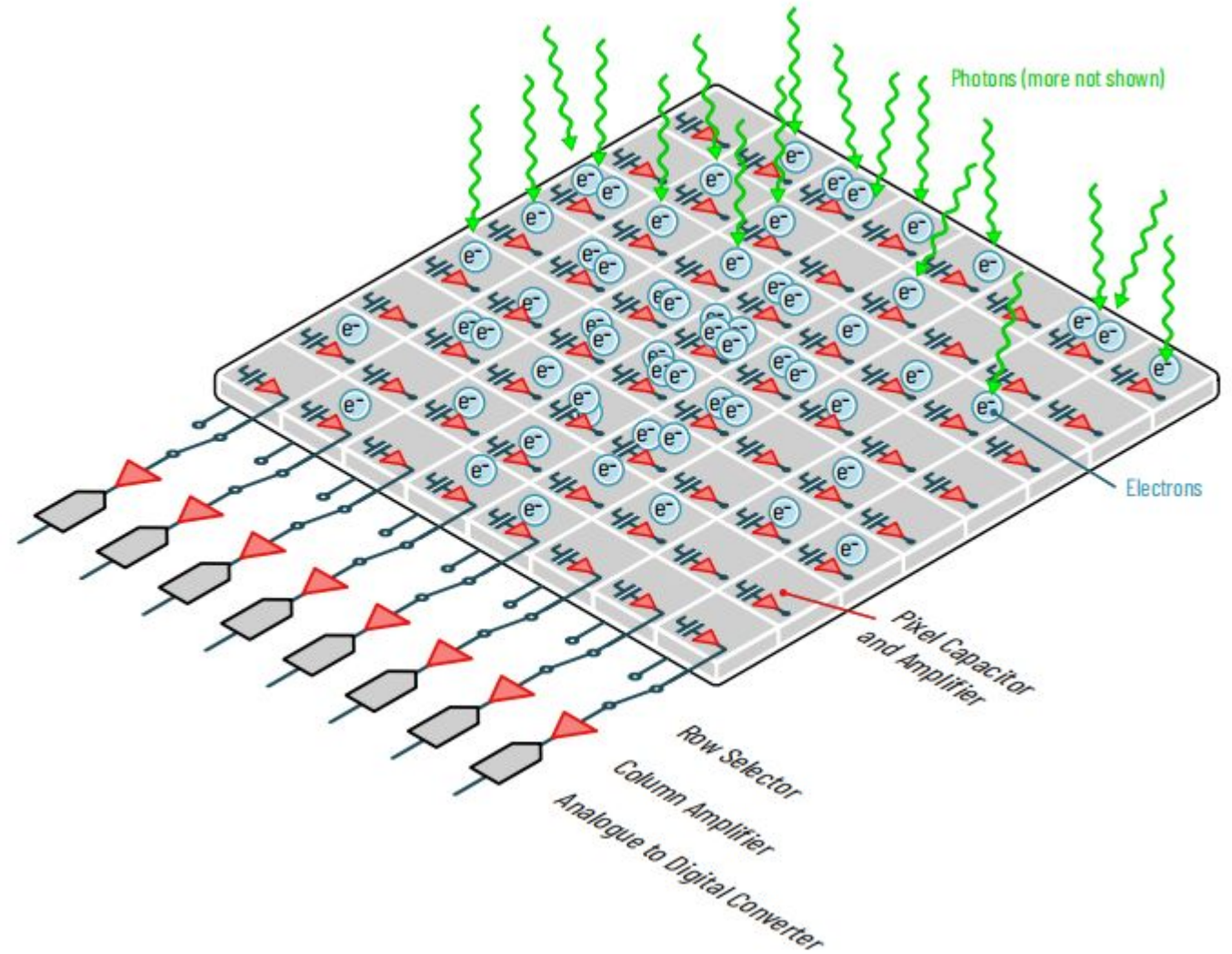
There can be additional storage arrays below the photosensitive array as shown in these two diagrams. The masked storage location is not sensitive to light and so when quickly transferred from photosensitive region, the mechanism reduces the chances of noise contamination in the storage area.

Sensor signal conversion (Optional)

CMOS

The main difference between CCD and CMOS technology is that in CCD, the electrons are converted to voltage, one pixel at a time. In CMOS, the conversion technology is embedded within each pixel. So instead of waiting for the sequential call, the pixel sensors can directly convert electron measurement to voltages. Then the processor just needs to read the data somehow. Generally, the data is read per row starting from the top. All the pixels in the row are read at the same time and passed through column amplifier and column ADCs.

Notice the column circuitry in the picture. Its absent for row because we read the rows one at a time. Once a row is selected, (in the picture the last row is selected) the column circuitry connects with the pixels in the row and concurrently reads all data.



In the current era, CMOS technology is more popular than CCD technology. In the earlier stages, however, CMOS was quite noisy owing to the local characteristics of the electron to voltage circuits. Additionally, in CMOS implementation you have to leave a certain amount of space to accommodate the conversion circuitry per pixel. So your photosensitive receptors need to be reduced in size. With improving technologies, we have more or less overcome these issues and owing to CMOS being way faster (parallel reading), it is being adopted more and more.

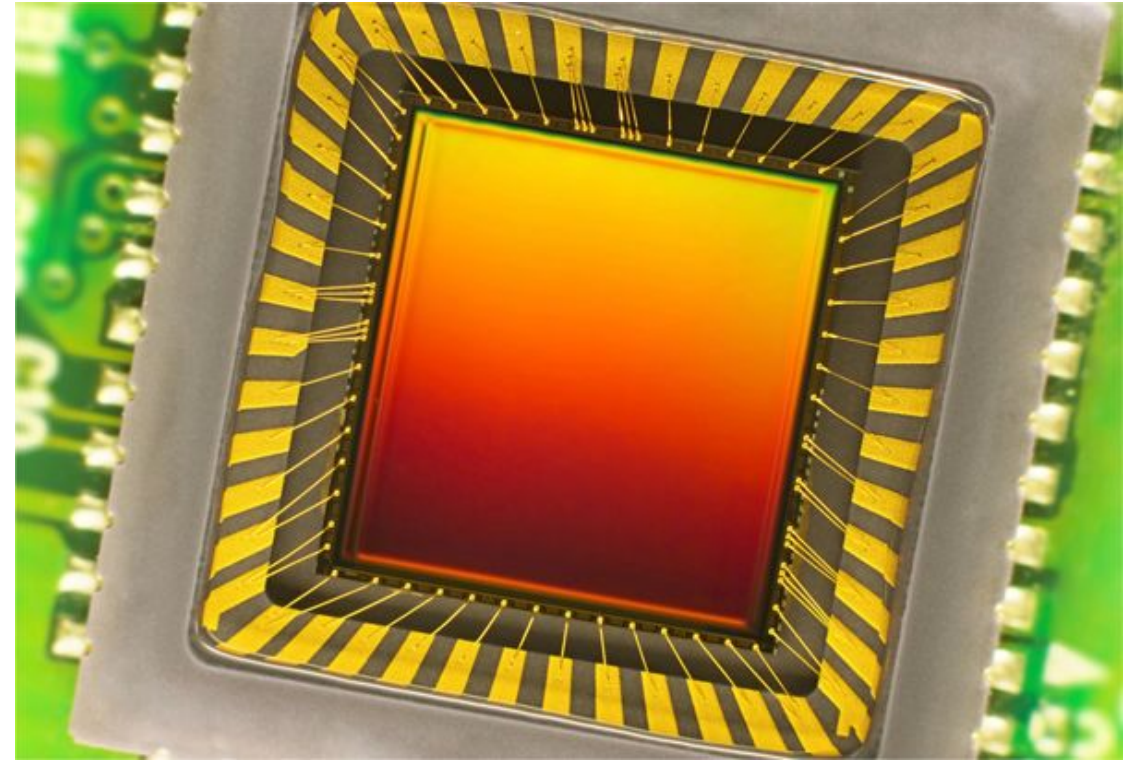
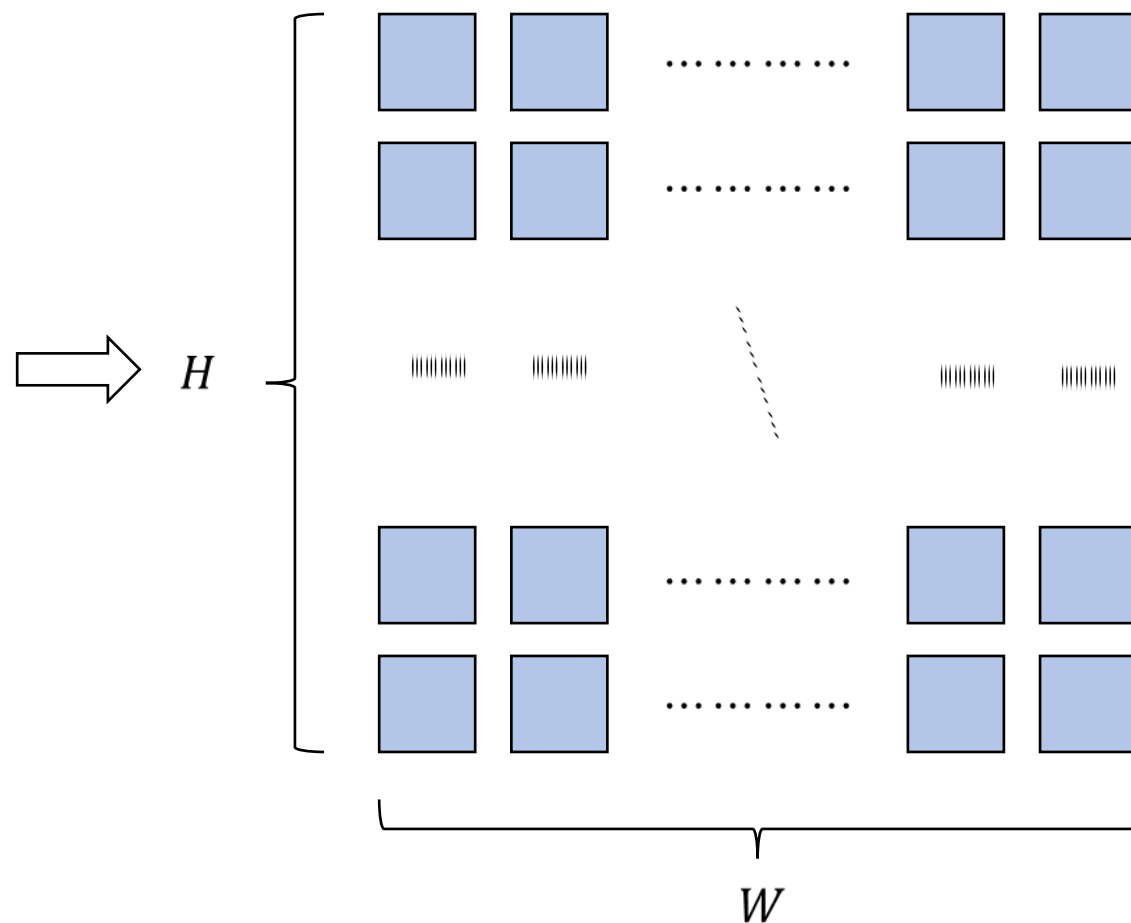


Image formation in the camera

Continued

With our knowledge now, we should be able to design a simple camera. Lets assume we have a custom sensor array using which we are trying to capture and image.

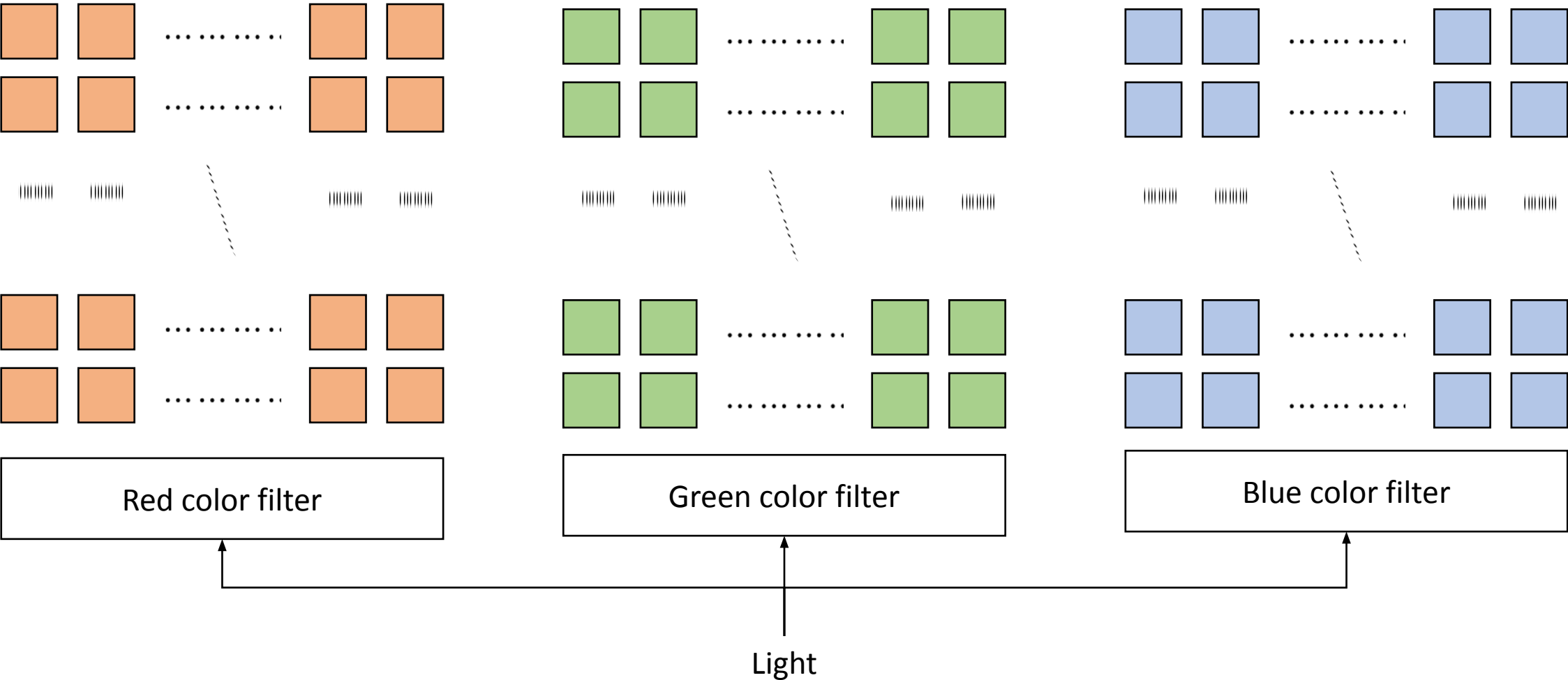


Lets assume for now that all type of camera internal processing (electron to voltage, amplification, ADC etc.) has been handled for you. When you went ahead and viewed the image, you found that the image turned out to be a grayscale image.



The sensor array captured the light details, brightness and darkness without any problem. However, you never designed the camera to learn how to distinguish between colors. So, it treated all colors as the same – overall brightness. So, to integrate color, we have to build separate sensor arrays just like we have separate cones in our eyes.

The easiest solution is to split the incoming light into RGB colors using color filters and then direct them to dedicated sensor arrays. In the figure the color of the sensor array doesn't mean anything special. All of them are standard sensors.



This gives us the standard 3 matrix formulation of RGB images, where each matrix holds the intensities of respective colors.

Red channel



Green channel



Blue channel



$$\begin{bmatrix} 1 & 0 & 3 & \dots & 39 & 32 & 18 \\ 6 & 4 & 0 & \dots & 13 & 9 & 20 \\ 6 & 3 & 2 & \dots & 28 & 25 & 18 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 223 & 205 & 201 & \dots & 8 & 7 & 4 \\ 209 & 210 & 213 & \dots & 4 & 3 & 2 \\ 218 & 217 & 223 & \dots & 3 & 2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 2 & \dots & 44 & 33 & 17 \\ 4 & 4 & 0 & \dots & 13 & 5 & 19 \\ 5 & 3 & 2 & \dots & 29 & 24 & 19 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 189 & 170 & 156 & \dots & 4 & 3 & 0 \\ 174 & 172 & 178 & \dots & 3 & 1 & 0 \\ 187 & 187 & 193 & \dots & 3 & 0 & 1 \end{bmatrix}$$

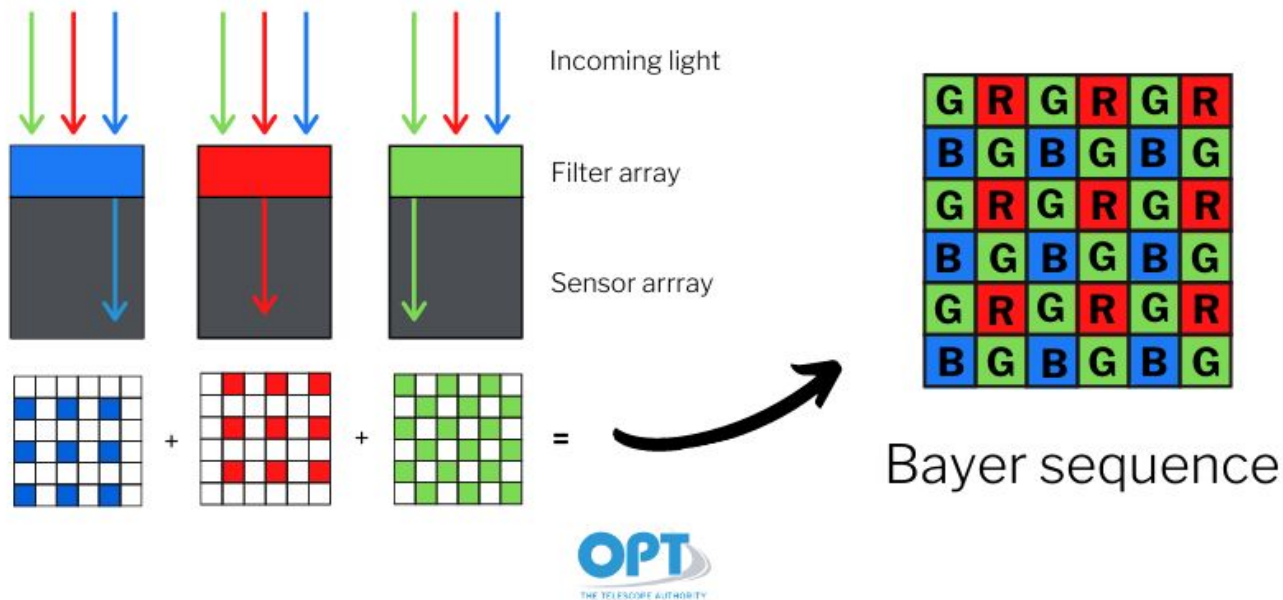
$$\begin{bmatrix} 1 & 0 & 0 & \dots & 38 & 28 & 13 \\ 5 & 4 & 0 & \dots & 11 & 2 & 15 \\ 3 & 3 & 4 & \dots & 24 & 20 & 14 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 128 & 106 & 97 & \dots & 3 & 4 & 1 \\ 108 & 110 & 110 & \dots & 1 & 2 & 1 \\ 122 & 117 & 131 & \dots & 1 & 1 & 1 \end{bmatrix}$$

The channel themselves are not colored as you might have expected. The colors are formed through tweaking display LED brightness. We will talk about it soon.

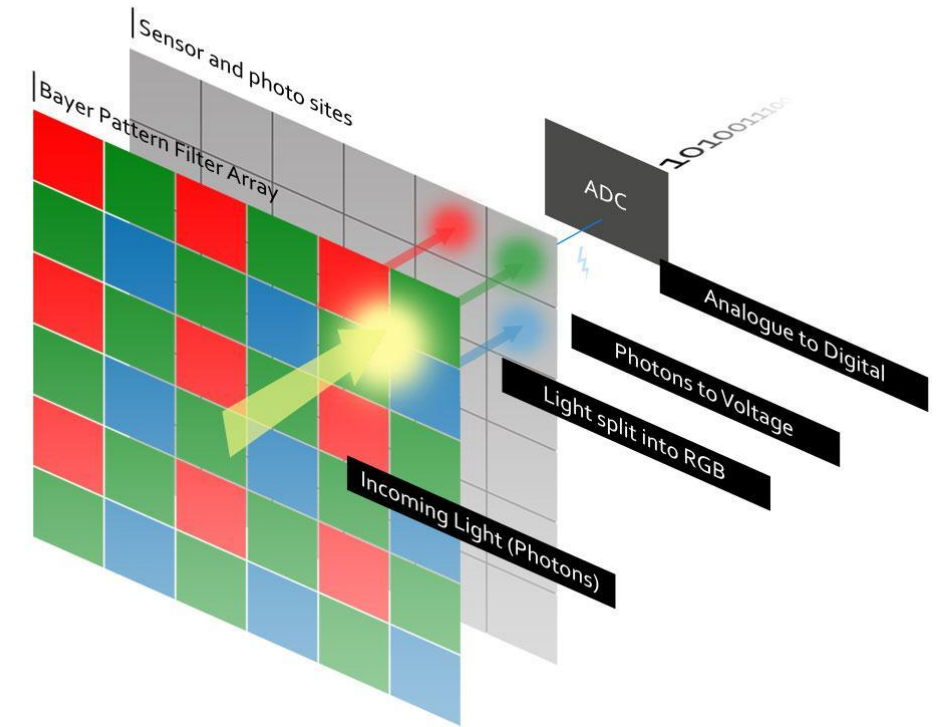
While the three channel separate sensor array may seem like a simple solution, it increases space requirement by 3. So instead a different approach is used for RGB imaging. We use one single sensor array and we follow a checkered pattern of color reading as shown below. The checkered pattern shown below is called the Bayer filter or the Bayer sequence. The final image is generated after matrix interpolation. Do note that other patterns, aside from Bayer, also exist but we will not discuss about them. Also note that the filter design now is super complex owing to how small each pixel filter needs to be.

Bayer Filter System

Color Astrohography Cameras



Camera Imaging Sensor with Bayer Filter

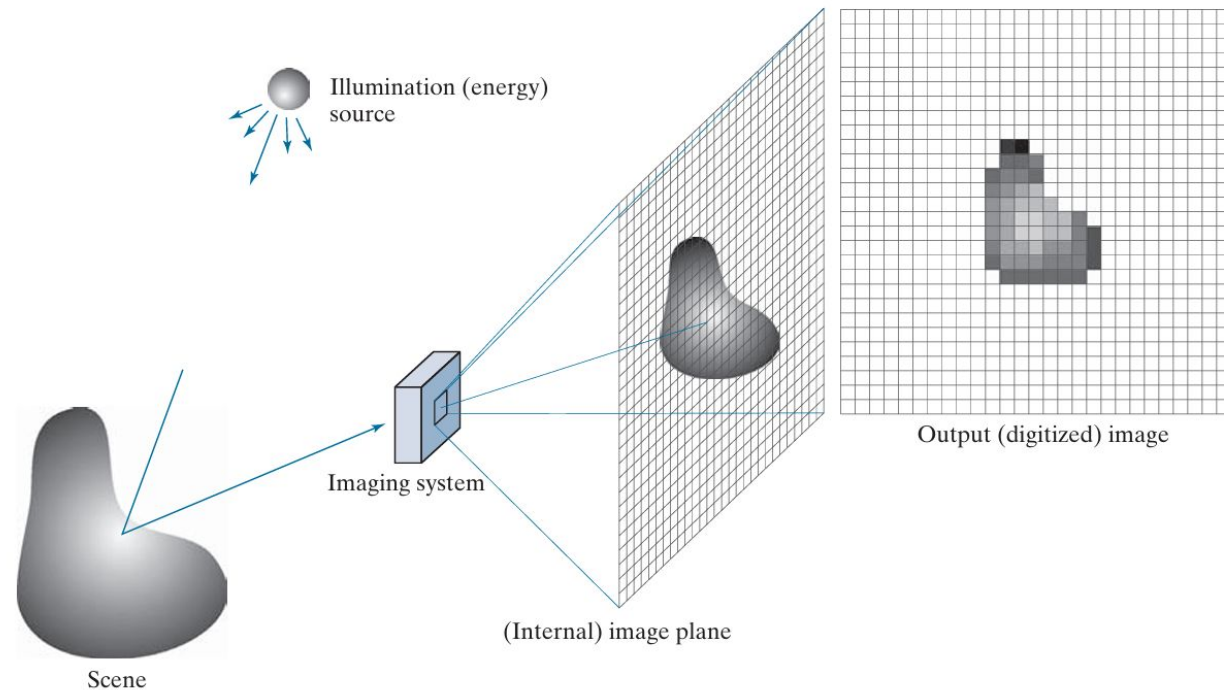


We will learn interpolation in a later slide.

Image storing

Sampling and quantization

The objects that we see in real life are continuous in nature. As in, no matter how much we zoom in, we can still find continuity in the object. Maybe we cannot always see the extent of continuity with our eyes, but it is there. However, when we capture an image of the same object, it is physically not possible to ensure nature's continuity. Look at the example provided below.



Depending on how many sensor pixels you have, you can ensure a smooth representation of the object you are trying to capture. In this particular example, the number of sensors is low and so you get a low resolution (blocky) representation of the image. Even with very high number of image sensors (high resolution), if you zoom enough, you will notice the blocky representation. This blocky representation is essentially a discontinuity, and the image is an approximation of the continuous scene that you are trying to capture. This discontinuous representation is called a sampled variant of the continuous scene and the process of making the scene discontinuous is called sampling.

You may have heard of sampling in other aspects. For example music is recorded at 44.1 kHz sampling rate. For our course, sampling is not very important. Additionally, sampling in image formation is easier to understand as it comes from the camera's hardware limitation and not directly from storage limitation. Even if you want, you cannot stack too many sensors in a fixed sized camera. To ensure full continuity, you have to place infinite number of sensors along both the height and the width of the sensor plane, which is physically impossible. And as such we get limited resolution images.

While most cameras give you ample resolution to work with and so we generally do not get to see the effects of under-sampling or extreme resolutions. We can resample existing images to give you an idea as to how under-sampling can add artifacts to images. See the example below.

800 x 800



400 x 400



200 x 200



100 x 100



50 x 50

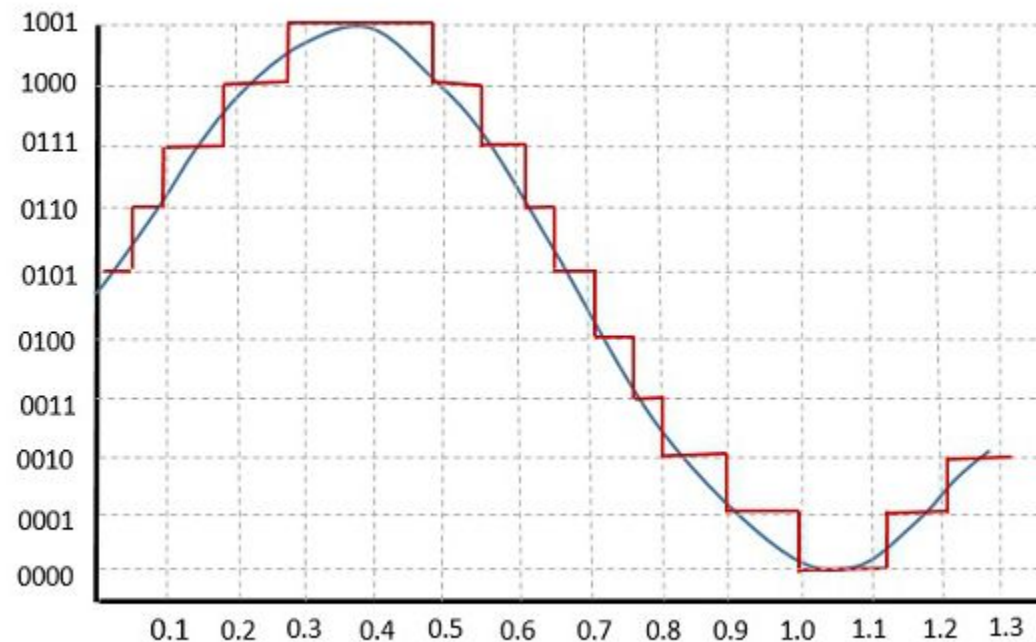


25 x 25



While sampling gives a discretized view in space, its not enough. We need to store the data in bits and an image with pixel values coming directly from real numbers will take infinite number of bits to maintain precision. We cannot store such high precision data and so we limit the number of bits. The most common format is 8 bit. Higher quality images can store in 10 bit, 12 bit, 16 bit or even 32 bit. For simplicity, in this course we will mostly work with 8 bit quantized images.

Essentially during quantization a range of data is collapsed to a single level, as shown in the graph (1D quantization). If you increase the number of bits, the data representation will be finer. But in most cases, we as humans cannot distinguish after a certain bit depth (number of bits used for representation). This limit varies with images.



Lets see the effect of varying bit depth on an image just like we did with sampling.



8 bit



7 bit



6 bit



5 bit



4 bit



3 bit



2 bit



1 bit

Image visualization

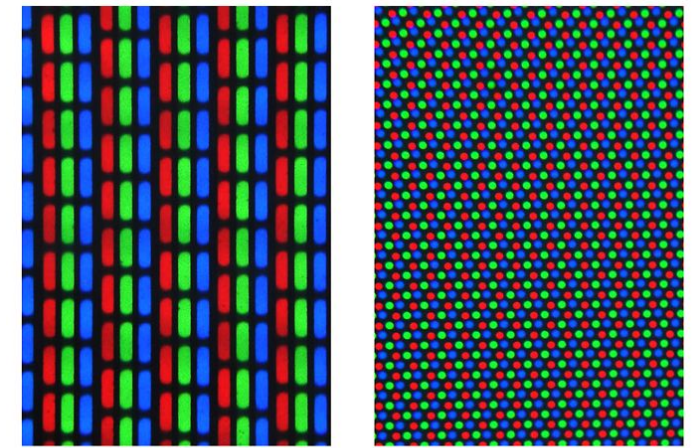
Different types of displays

There are mostly 3 types of monitors –

1. CRT (Cathode ray tube) (redundant box monitors)
2. LCD and LED monitors (presently most common)
3. OLED monitors (gaining traction)

While their working principles vary, they stack RGB colors in very close proximity (either through emission, or filtering). These colors when viewed from afar, feels like one solid pixel (limitation of human eyes). How intense each color should be is determined by the stored intensity matrices (essentially our quantized color data).

Not all monitor can support bits higher than 8 bits. This is something that will come into play during HDR image formation. Some do support 10 bits or 12 bits. For example OLED is generally 10 bits.



21" TV CRT Display

17" PC CRT Display

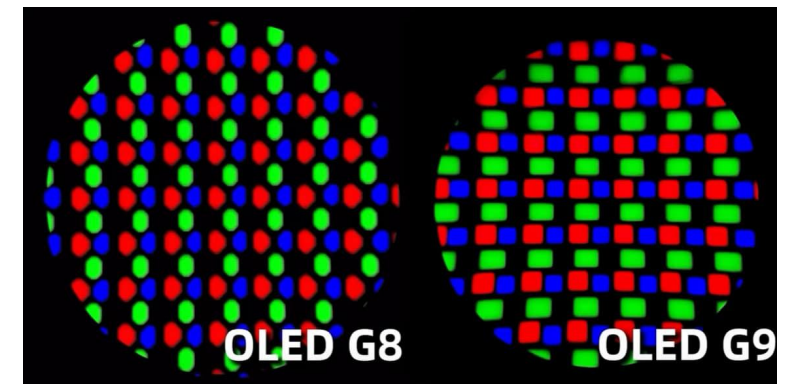
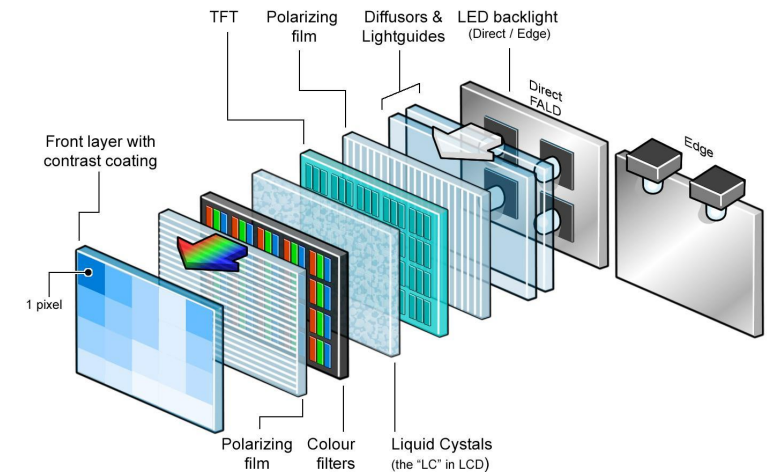


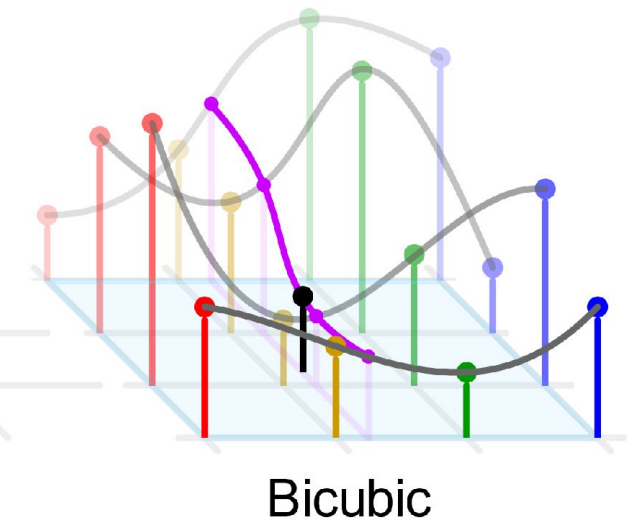
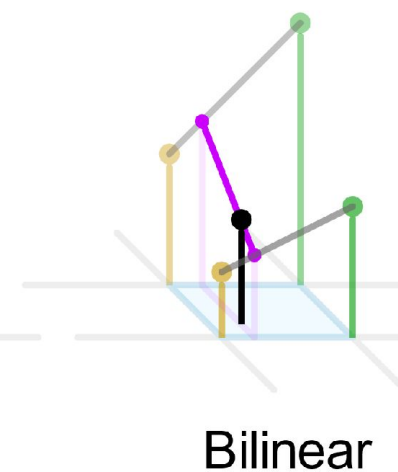
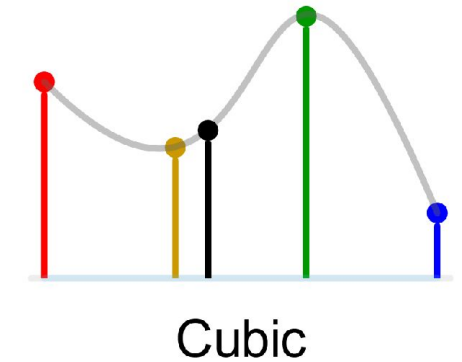
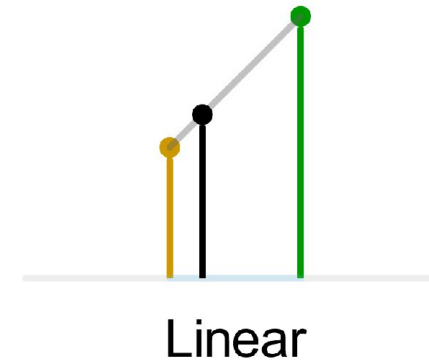
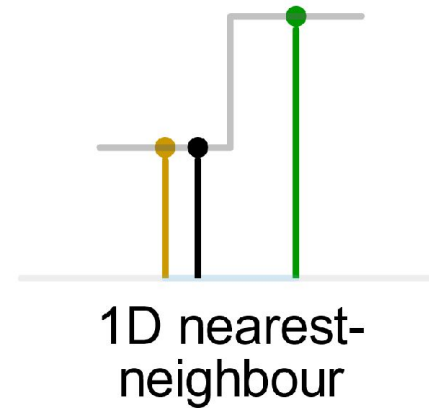
Image rescaling

Upscaling, interpolation and resampling

Now let's assume our device gave us images that are low resolution and stored with low bit depth. Can we do anything about it to fix it? We cannot fix low bit depth, but we can improve resolution through interpolation. There can be different types of resampling, but the most common three are shown here in this slide.

We will mostly focus on nearest neighbor (2D) and bilinear interpolation. Bicubic interpolation is difficult to perform by hand and so we will not cover that.

Do note that interpolation in itself is not exactly easy. Let's look at simple cases. Then we will understand how it is actually done through a small example.



1		3		5
13		15		17
25		27		29

Assume a 3x3 image is being
interpolated to 5x5

1	1	3	3	5
1	1	3	3	5
13	13	15	15	17
13	13	15	15	17
25	25	27	27	29

Nearest neighbor
interpolation

1	2	3	4	5
7	8.5	9	10	11
13	15	15	16	17
19	20	21	22	23
25	26	27	28	29

Bilinear interpolation

For the implementation of nearest neighbor algorithm the ties were broken through priority. Top and left received more priority. You can use different priorities

For bilinear interpolation, at first fill up the rows and columns directly beside the known values using linear interpolation, essentially $\frac{a+b}{2}$. Then fill up the diagonals with the help of already interpolated values. You can approach from any direction while filling up the diagonal spaces either through the row values or column values. Try it out and see if you get the same results.

Whatever was shown in the last slide on interpolation is not exactly accurate for general cases. It was a selected structure that avoided units outside of the range (extrapolation) and also essentially upscaled the original image by a factor of 2. In general cases, the interpolation is done through a method, colloquially called resampling and is a little difficult. Lets look at a simple example -

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

Original 6x6

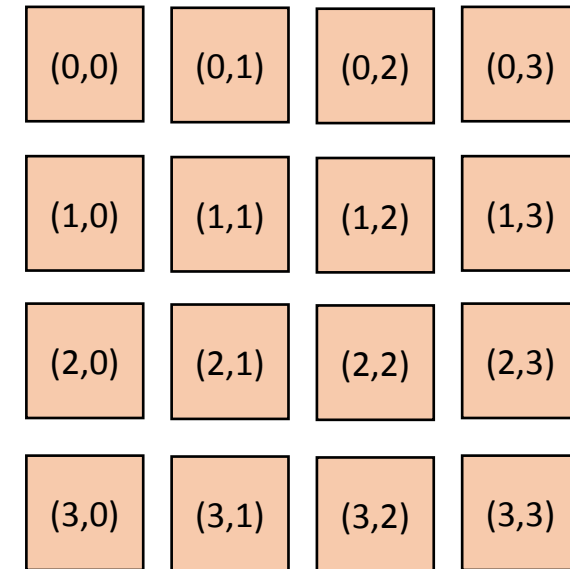
(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

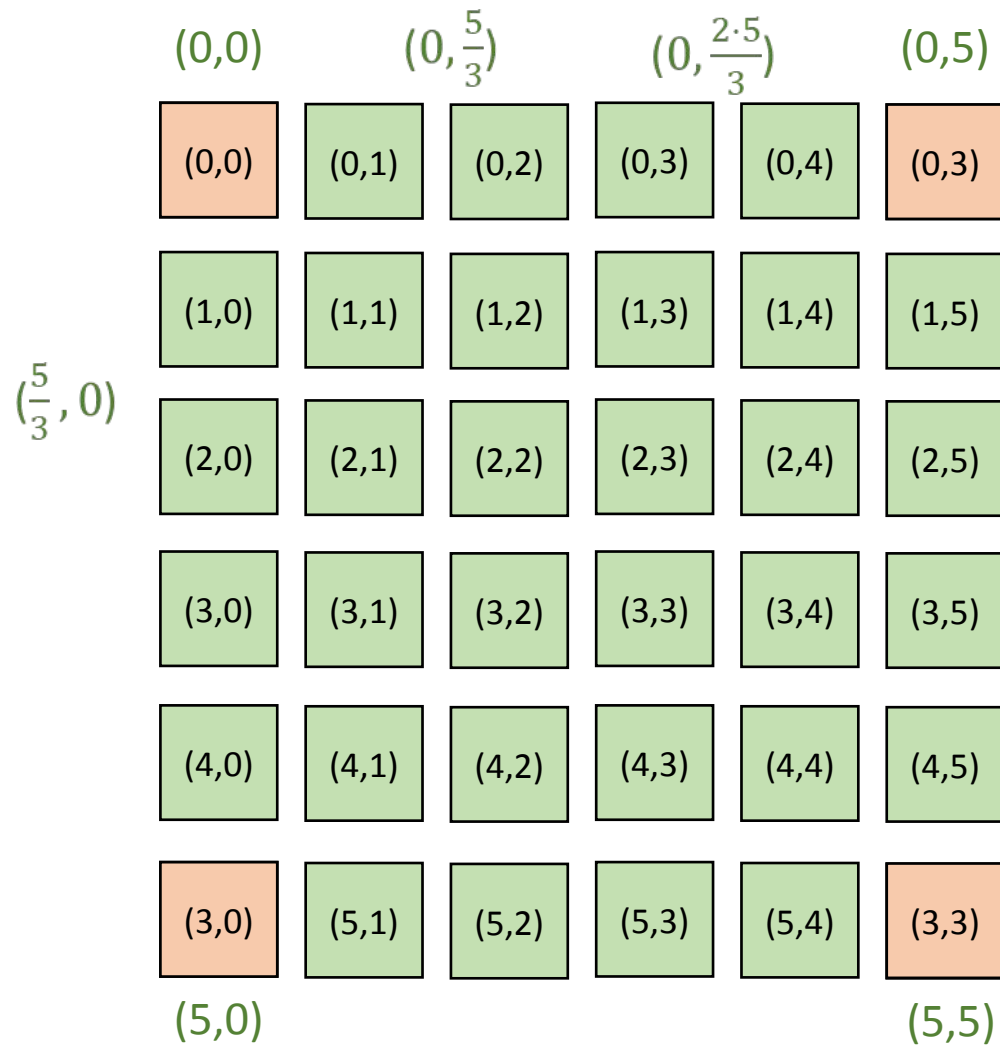
To be interpolated (4x4)

There are two variants where end points are either aligned or they are not. We will go for the simpler case where the end points are aligned and then an equivalent coordinate mapping is done from the original interpolated indices to the source indices. Lets see how it is done in the next slide.

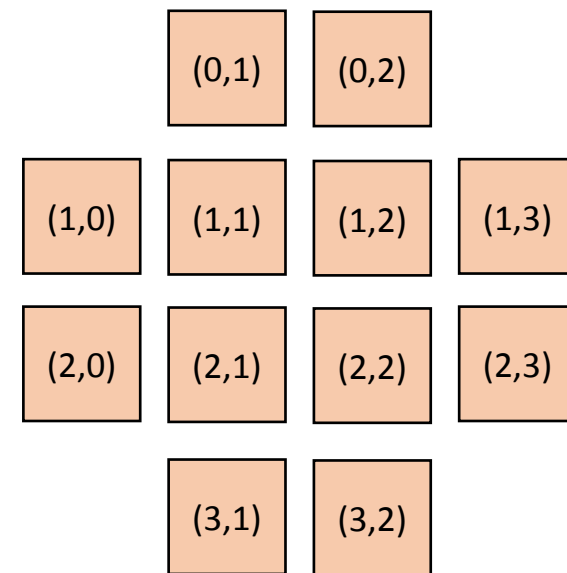
Note – Although this is an example of downscaling, the same mechanism works for upscaling too.

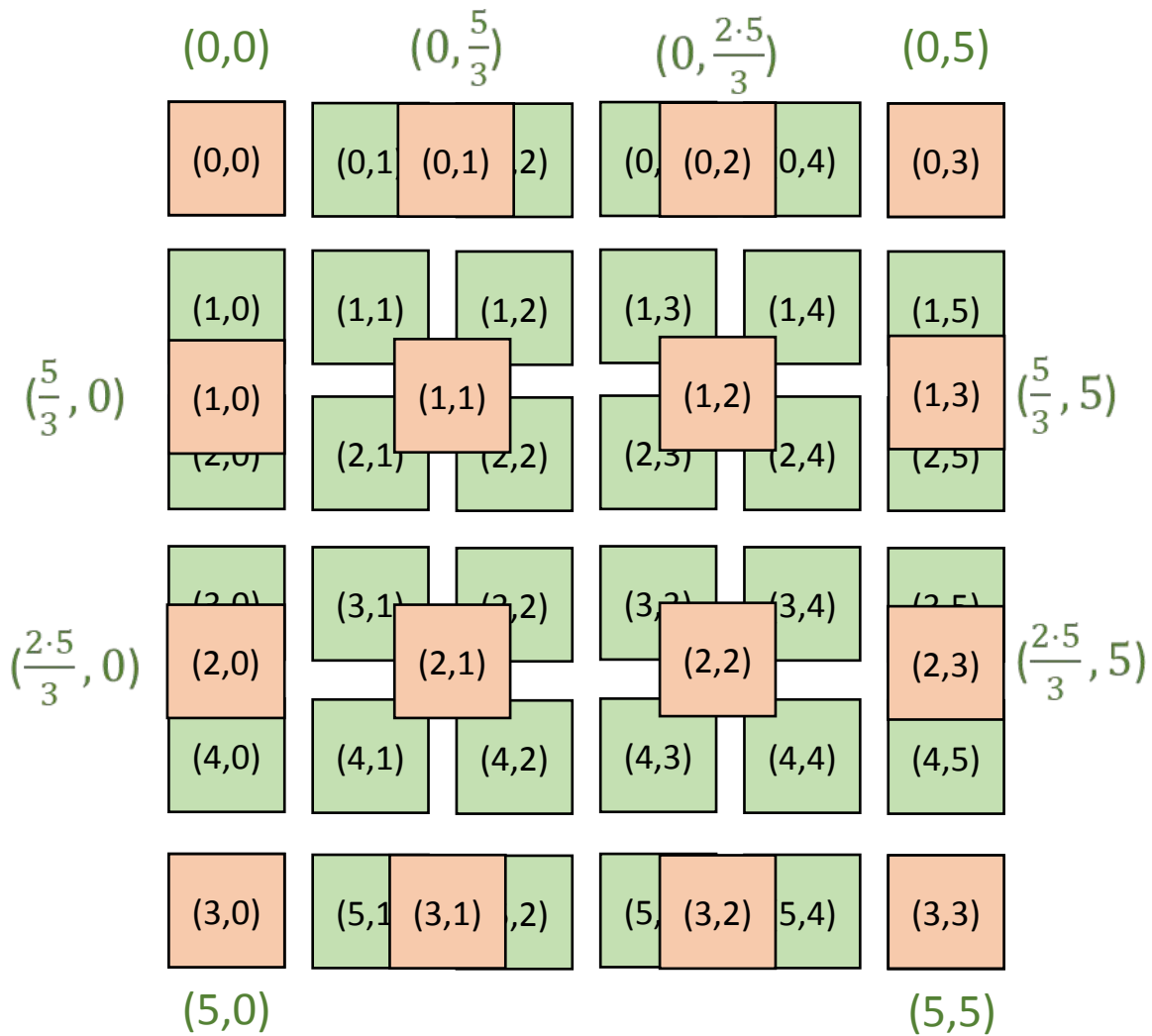
For each step in interpolated x-axis, $s = \frac{5}{3}$
step is covered in original axis.



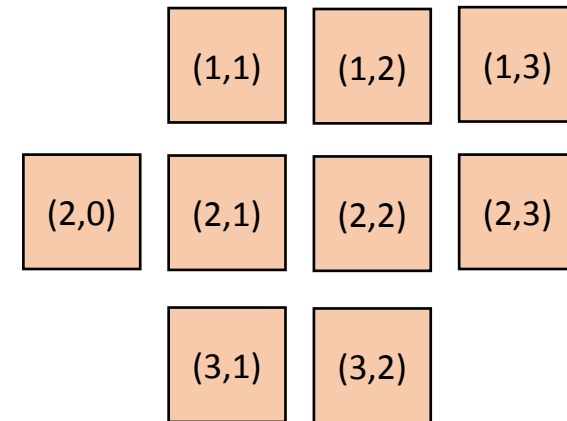


Continuing in the same pattern,
we get something like ...

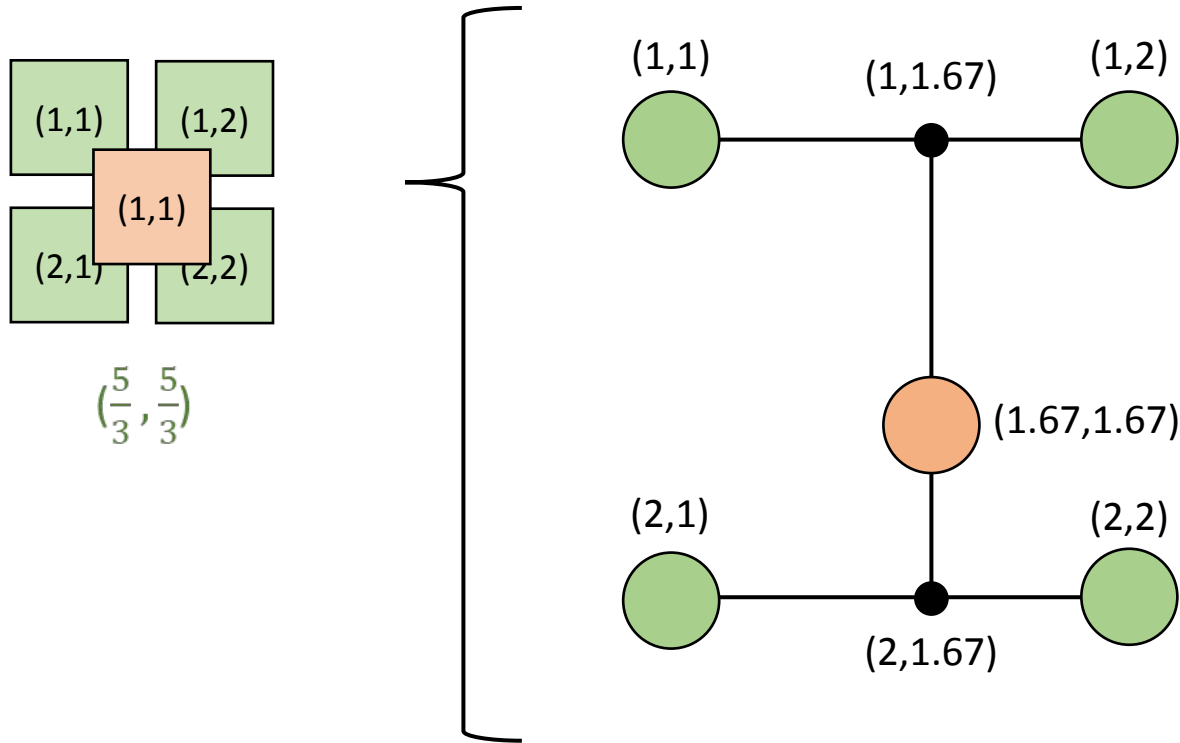




Continuing in the same pattern,
we get something like ...



Now question is how do we interpolate? Lets take an example. Lets interpolate the (1,1) block using bilinear interpolation.



At first, find (1, 1.67) and (2, 1.67) using linear interpolation using (1, 1), (1, 2) and (2, 1), (2, 2) respectively.

Then use (1, 1.67) and (2, 1.67) to find out (1.67, 1.67) using linear interpolation.

You can also first solve column-wise and then go row-wise.

Essentially, resampling isn't easy unless you are given some very specific cases.

Dynamic range
In human visual system

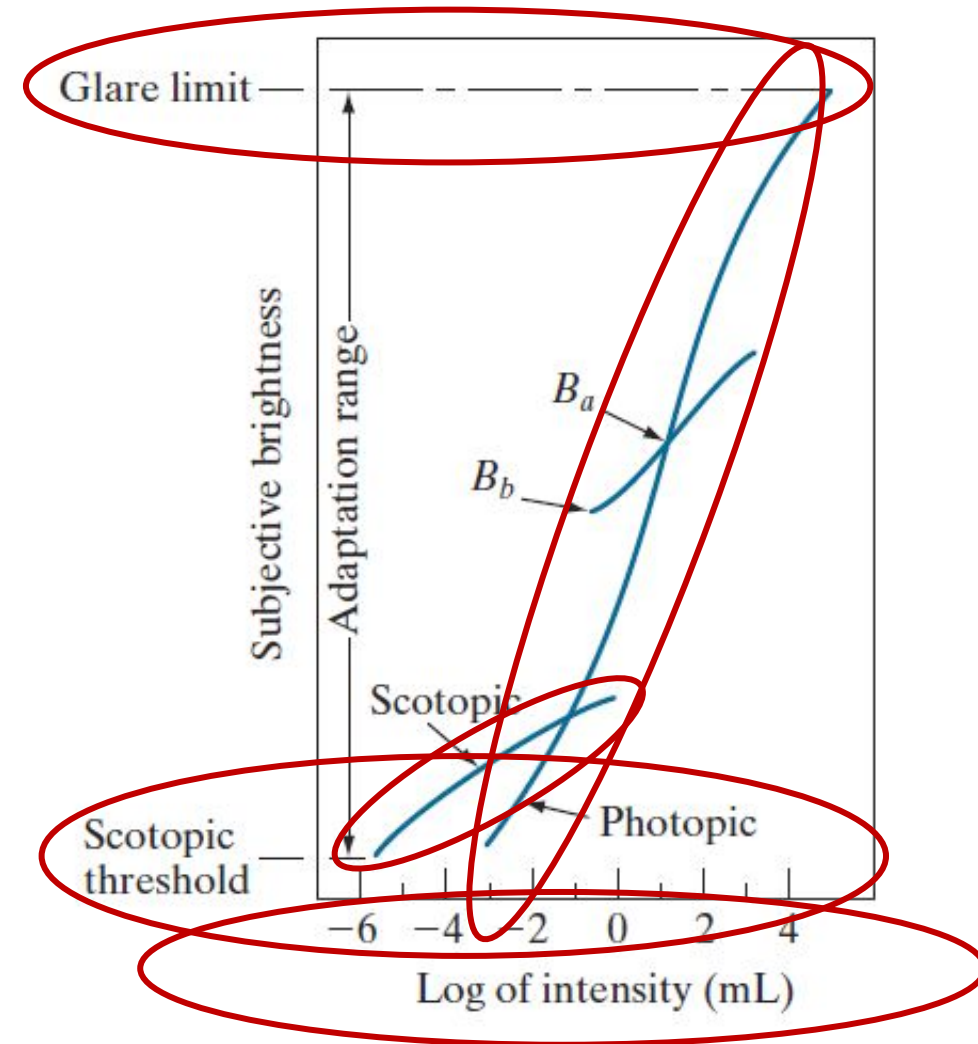
The graph provided on the right may be a little bit difficult to understand by itself, but it contains a lot of nuances about human visual systems. Let's first understand the graph and then we will talk about dynamic range.

The two marked levels are the limits of human visual system. The glare limit is the maximum brightness that your eye can handle and the scotopic threshold is the least brightness that you can see.

Notice that the true light intensity is given in log scale and the graph as a result becomes somewhat linear, implying that human subjective brightness has a logarithmic relationship to true brightness.

The small highlighted graph is perceived brightness in the scotopic region (rod-based vision in low light). Notice that this graph is quite small and is limited in both brightness axis. This is because if brightness is high, your eyes automatically switch to photopic vision (cone-based vision in sufficient light)

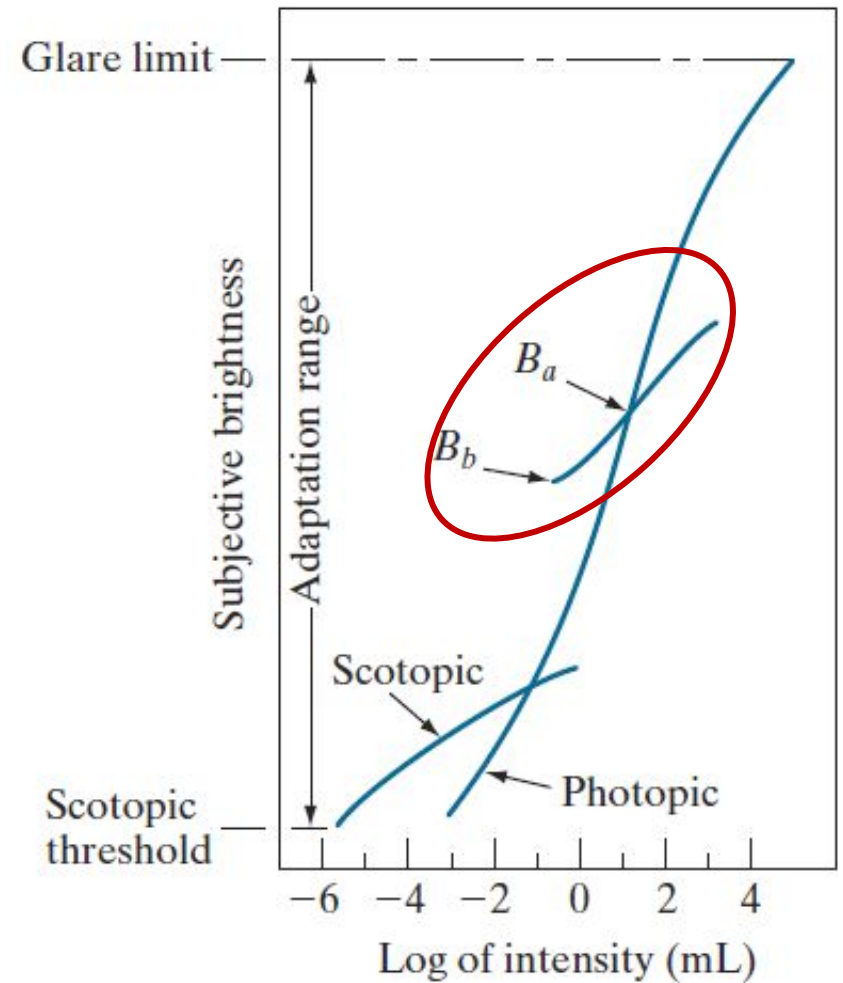
Ignore the small line segment for now. The big line segment is the range of brightness that humans can perceive under photopic vision. It is clearly larger than scotopic vision.



Now let's finally focus on the small line segment. While the human visual system can see a huge range of brightness, as viewed by the photopic vision graph, it cannot utilize the full range at a time. It needs to adapt to a certain brightness point and it can then use a certain range around that adaptation point. In the graph, the small line segment is the local adaptation graph centered at B_a . In this adaptation range B_b is the least brightness that our eyes can distinguish. Anything below that becomes pure darkness.

Think of watching movie in a movie theater. Your eyes get adapted to the darkness. So when you come out, the sudden brightness irritates your eyes. Similar phenomenon can be experienced by staying outdoors in the sun and then coming to a shaded place.

Do note that the upper limit of the adaptation graph is not set explicitly as with increasing brightness the center point B_a will just continue shifting upwards until glare limit is reached. Also do note that the adaptation graph is also linear in log scale. It's like a miniature version of the complete photopic graph.

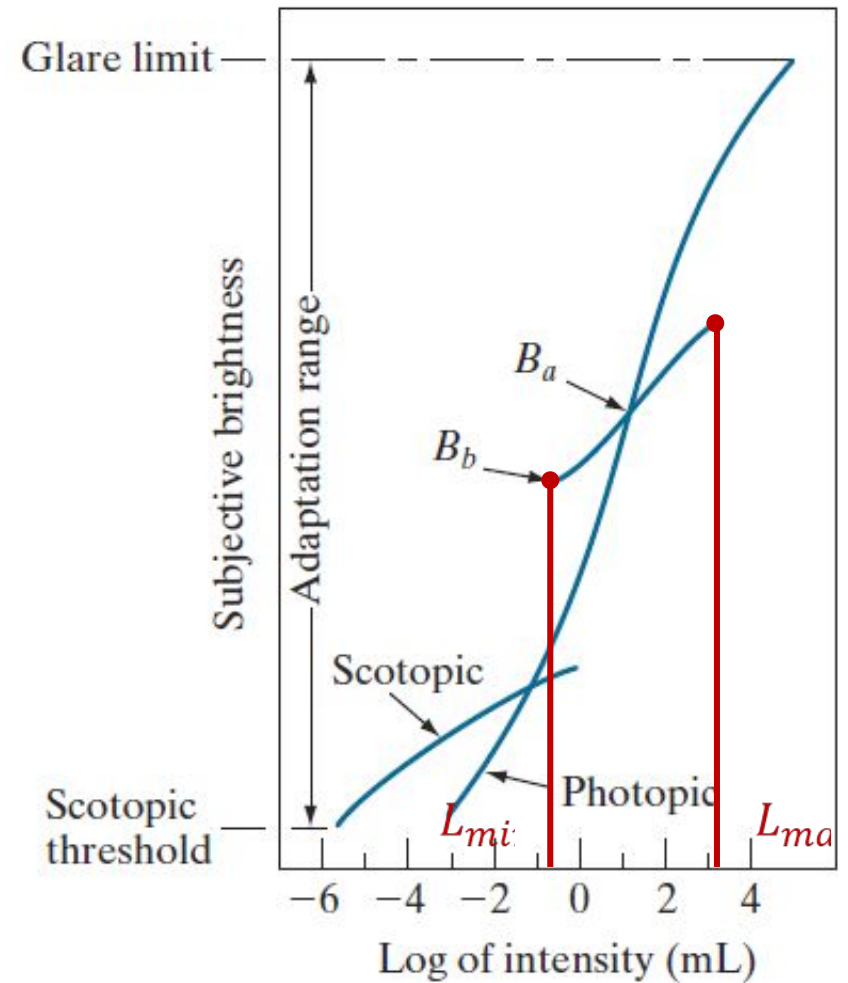


Now with the graph explained, we can focus on dynamic range. Dynamic range is essentially the logged ratio of the max brightness and the min brightness that can be perceived. In this case, when adaptation is centered at B_a , the dynamic range is given by $20 \log_{10}(L_{max}/L_{min})$.

FYI, scotopic vision actually has a higher dynamic range than that of photopic vision and as the ambient brightness increases, dynamic range decreases. But with increased ambient brightness your cones work better and you can notice minute details and changes better in a localized brightness setting.

Have you ever seen X-rays being read? The radiologists place the X-rays onto a light source before making observations. The increased ambient brightness allows them to understand minute changes in the X-ray plates.

This also implies that high dynamic range doesn't improve minute details. You are essentially getting too many details and not focusing on any specific one. Kind of a reverse effect.



HDR images

In cameras

Unlike the human eye, cameras cannot adapt to brightness themselves. They have a lot of moving cogs that help them to adapt. Two of the most important features that help a camera adjust to brightness are – ISO (the amplifier gain) and exposure time. The camera images are essentially just voltage levels based on how many electrons have been emitted. So accounting both ISO and exposure time we can formulate a simple equation –

$$V_{final} = I \cdot V(q(t))$$

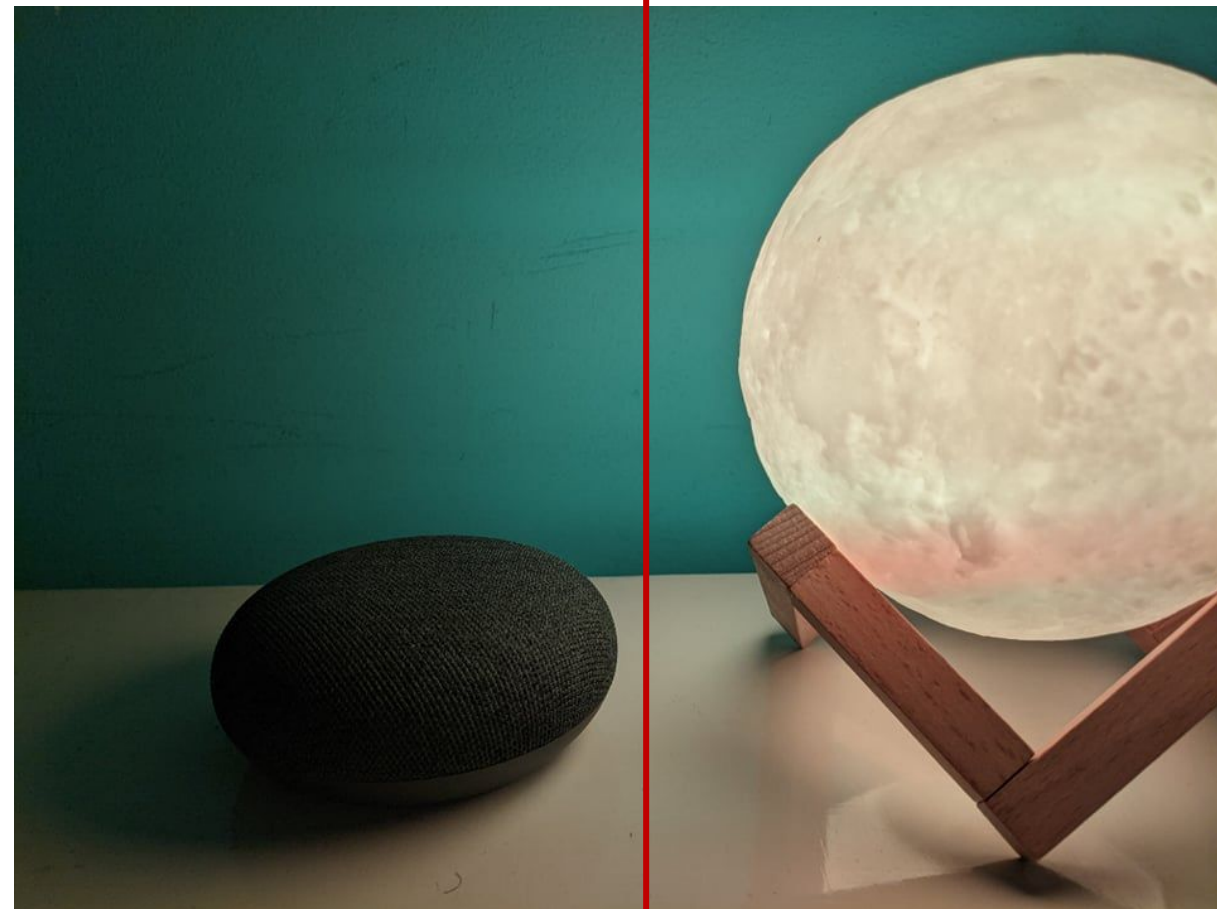
Here, $q(t)$ is the amount of electrons emitted by a particular photoreceptive sensor and it is dependent on time (the exposure time). The $V(x)$ is essentially a mapping function from number of electrons to initial voltage levels. The I is the gain or the ISO. Lets assume for simplicity $V(x)$ is a linear function and the ISO is fixed. Then the only factor deciding how to map brightness (via the displaced electrons) to voltage is the exposure time.

Lets try to understand how camera adjusts to brightness under our assumptions to this specific image. Lets assume the camera can read a min of V_{min} and max of V_{max} . Anything beyond that is physically impossible for our camera to read. Thus the maximum dynamic range our camera can have is essentially $20 \log_{10} V_{max}/V_{min}$. A camera can, however, localize a certain image in a smaller voltage range if needed which in turn can update the voltage/bit resolution and dynamic range adaptively (dynamic range cannot increase).

For the sake of easy understanding HDR images, lets assume the camera will always map brightness within $[V_{min}, V_{max}]$ and the voltage per bit remains constant. Additionally, for regular images, we will assume an 8 bit formulation where V_{max} is assigned 255 and V_{min} is assigned 0.



The image has two distinct bright and dark regions.



Bright region

Resultant image (only bright details remain)

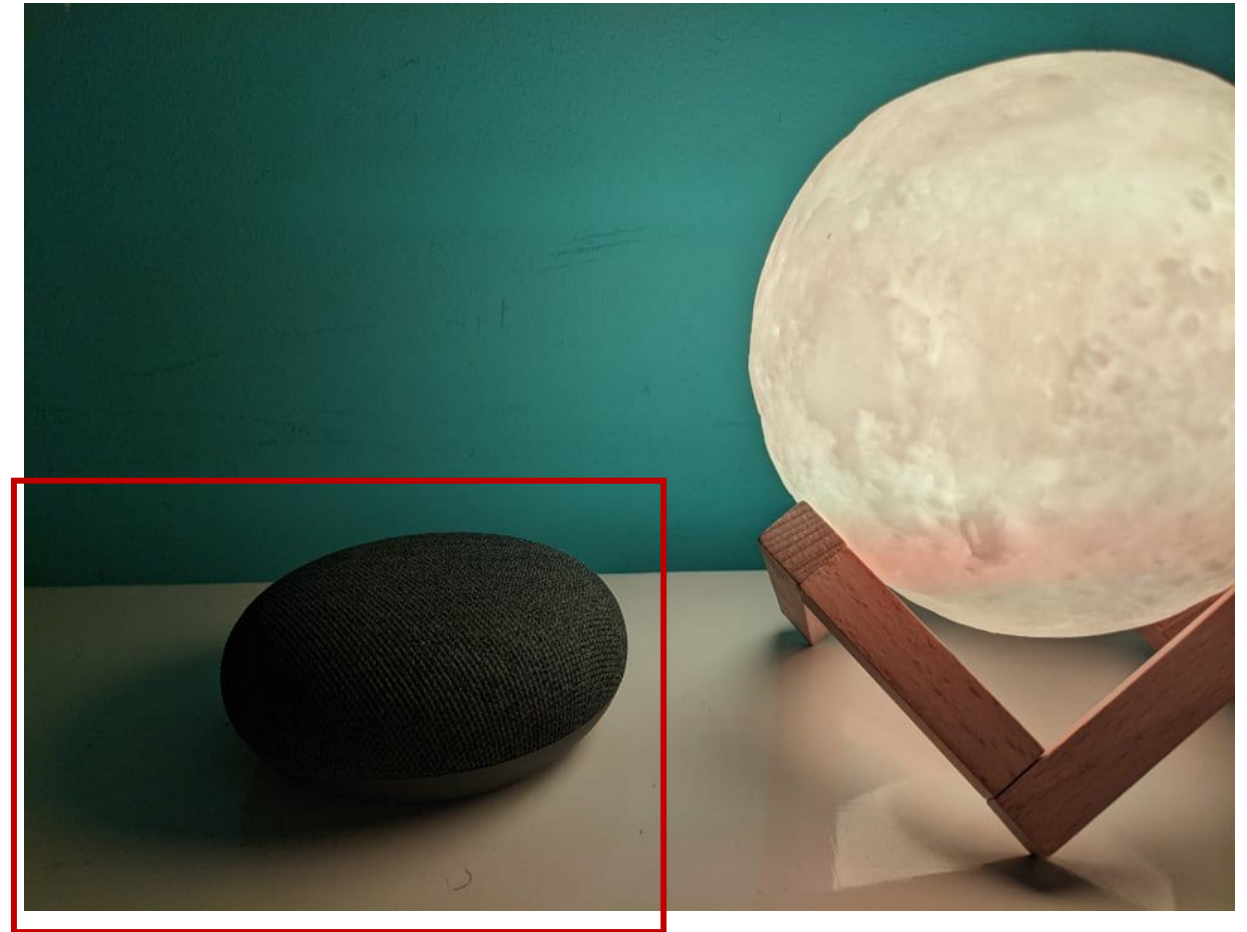
Turn on the slideshow.

Do note that the mapping functions have been created only through tweaking exposure.

The image has two distinct bright and dark regions.



Resultant image (only dark details remain)



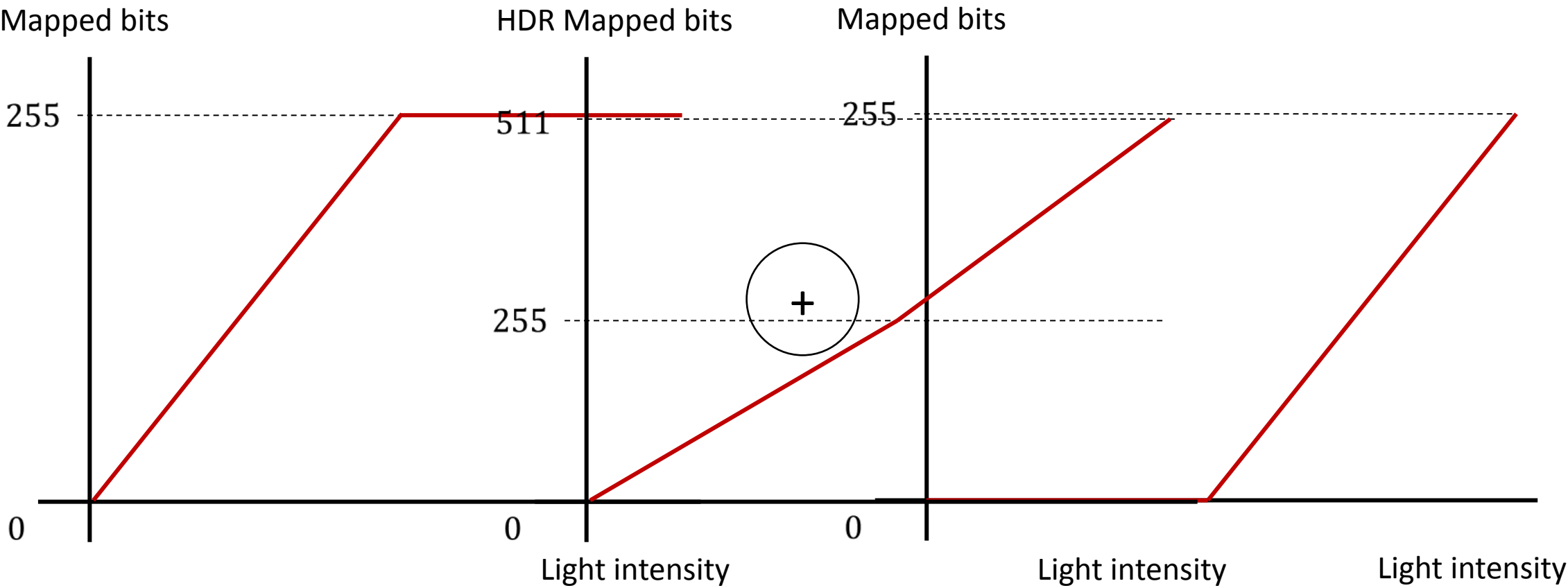
Dark region

Turn on the slideshow.

Do note that the mapping functions have been created only through tweaking exposure.

Now the question is, if we have reached hardware limitation and cannot accommodate both features in one image (low dynamic range), how can we improve it and how does HDR (high dynamic range) imagery work?

The simplest case is just stitching the two images (in our example) built at different exposure to create a new image. So essentially,



The merging case shown here is the simplest HDR possible. And it only accounts for 2 exposure states. In practice, HDR generally accounts for 3 to 5 exposure states. Also the stitching is not this straightforward. The timing of shots can create motion blurs (ghosting effect) that needs to be mitigated. Additional processing called tone mapping needs to be carried out too. But assuming everything is done properly (lets ignore the complexities for now), we get an HDR image that can retain all the brightness regions.



Also do note that HDR images are generally higher than 8 bit since they have to account for additional brightness levels.



SDR stands for standard dynamic range.

Why combining algorithms matter?

Look at the picture on the right. Since HDR stitches multiple pictures together, the time delay on the pictures can create ghosting effect for moving objects. Proper merging algorithms can mitigate this effect somewhat.

What is tone mapping?

Simply put, it's a complex set of transformations done to view a HDR image in bit limited settings. They are too complex for this course and so we will not cover them.

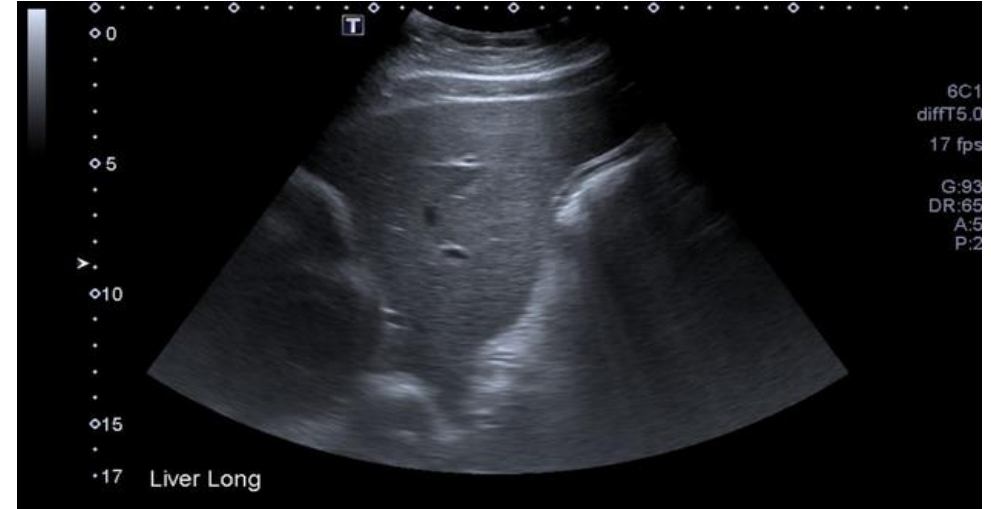
Without Tone Mapping
(simulated)



Other imaging formats

Not all images that are generated around us come from reflective imaging. Most medical field images are formed differently.

For example, ultrasound imaging is formed using sound (based on reflection through echo). MRI is formed by at first polarizing water inside tissues and then releasing the energy (emission imaging). X-ray imaging is formed through absorption mechanism. Bones absorb more rays and soft tissues let more rays pass through. Through wherever, more rays pass through become darker in the projection film (negative imaging).



Ultrasound image

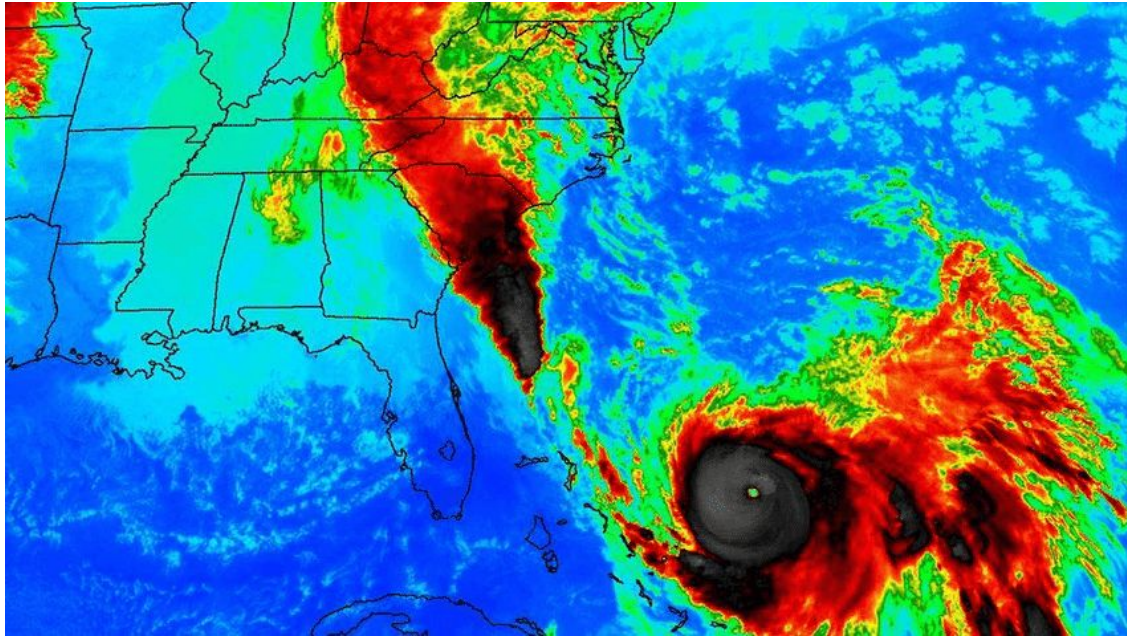


X-ray

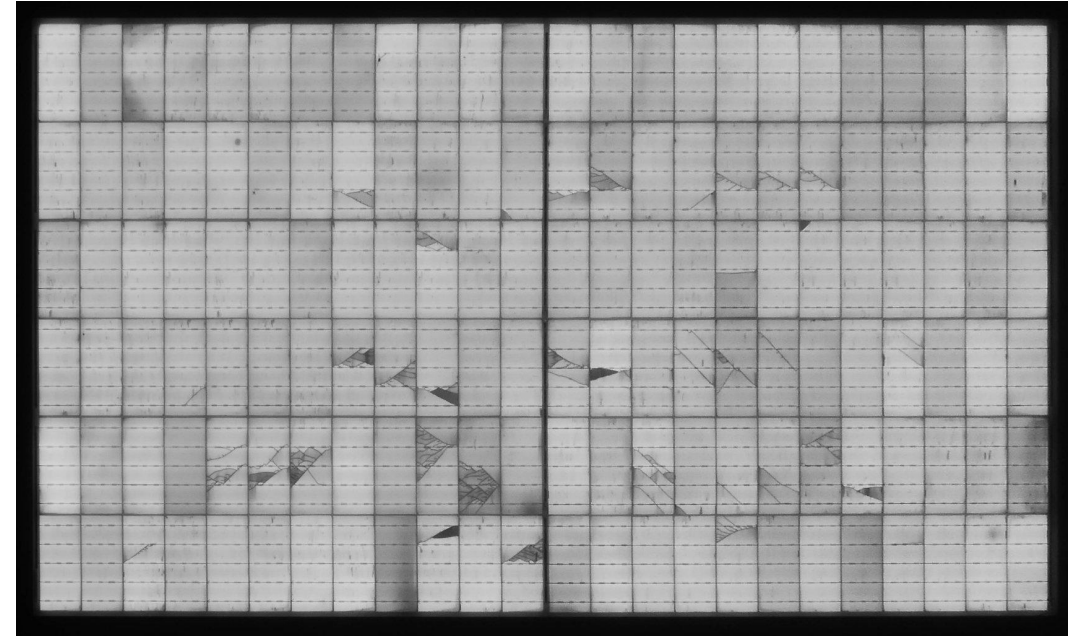


MRI

Aside from medical sector, infrared imaging is used in satellite imaging, Electroluminescence (EL) imaging is used to diagnose solar panels. So, while reflective imaging is probably the biggest source of generated images, it isn't the only source.



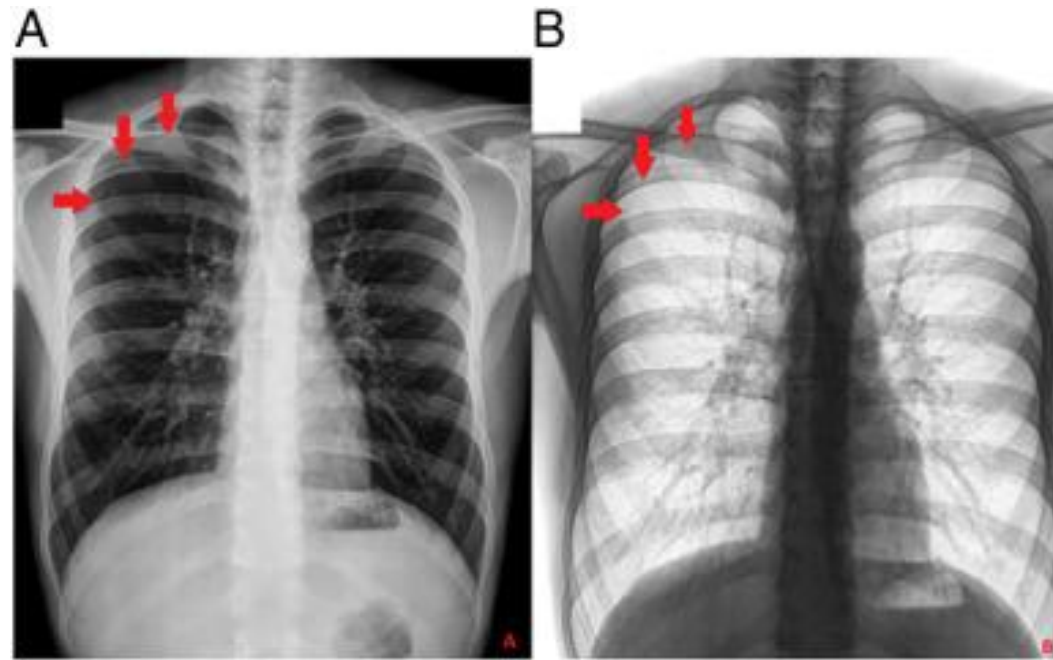
Infrared satellite image



EL image of PV panels

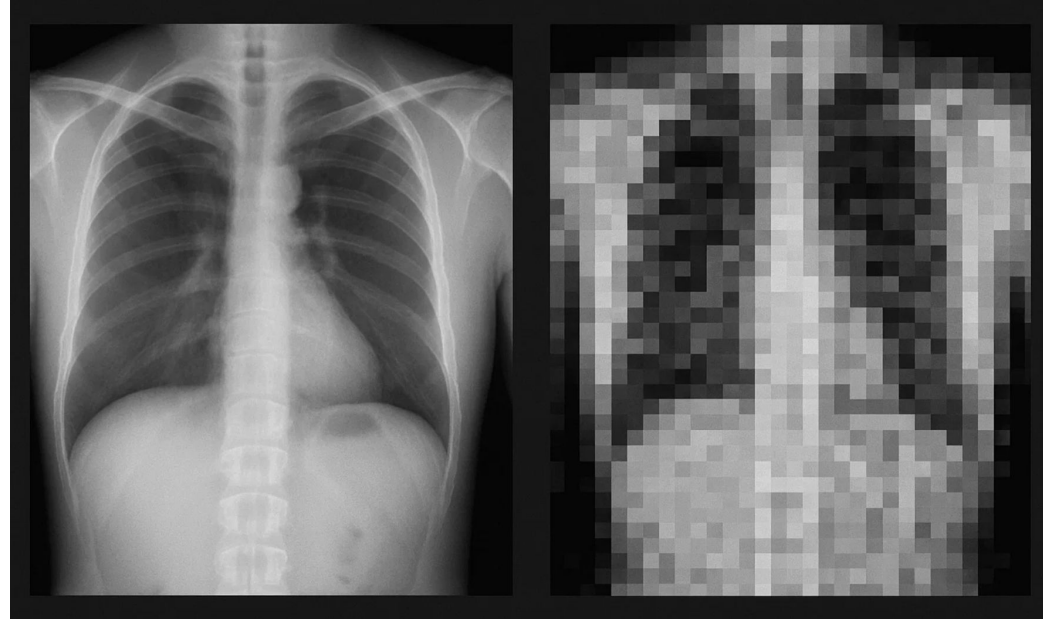
Questions

1. Ameet was trying to read X-ray images but was struggling to find details within them. Pratyasha recommended that he invert the image. Why do you think Pratyasha recommended image inversion? Explain from the perspective of photopic vision. For your reference, follow the picture provided below. **A** denotes standard X-ray and **B** denotes intensity inverted variant.



2. Briefly explain HDR imaging and the steps involved with the process.
3. Naqvi tried to record color a color scenery using his own sensor array camera. However, when he viewed the capture image, he found a grayscale image. What could have gone wrong? And how can he fix it?

4. Briefly explain how modern day cameras capture color details.
5. What type of artifact does the image on the right represent?

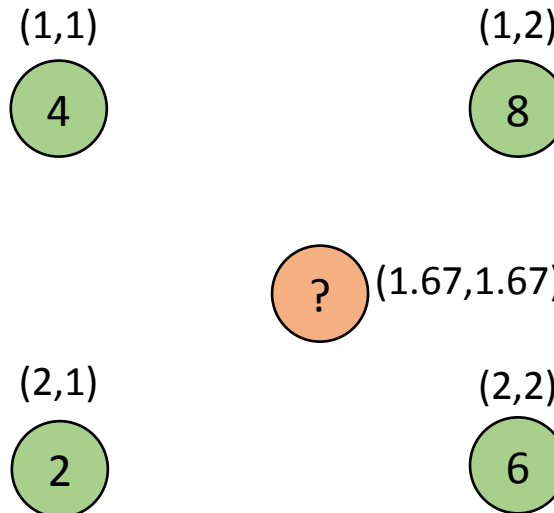


6. Abrar was playing a cricket match in the afternoon sun. After the match when he entered the academic building, he was momentarily blinded. Explain the phenomenon behind what happened to Abrar using relevant graph.
7. Briefly explain how colors stored as intensity matrices can be displayed.

8. The given beach image is hard to look at. What could have been done differently to make the picture better and soothing on the eye.



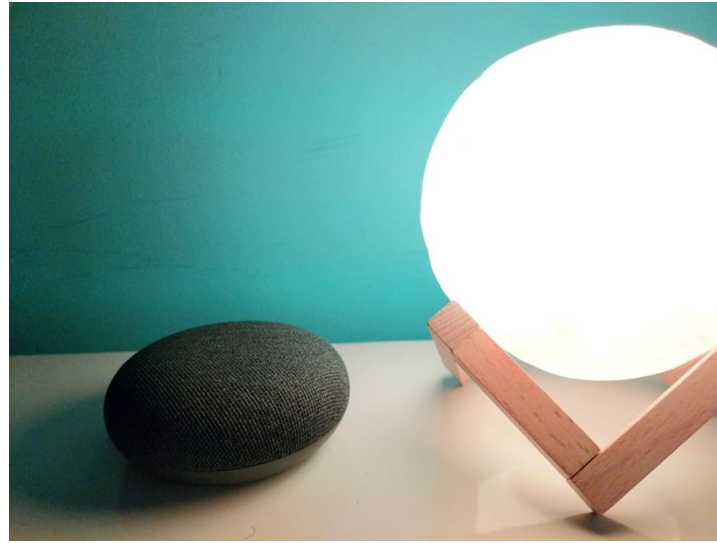
9. Interpolate the unknown value in the graph using bilinear interpolation -



10. Fill up the empty spaces with (a) nearest neighbor interpolation, (b) bilinear interpolation. Maintain precision up to one decimal place. For the last column and the last row, extrapolate. Do not go for the resampling formulation.

13		6		51	
15		7		54	
11		3		48	

11. Mark the image taken with a higher exposure time. Give reasoning to your answer.



12. What was done wrong during capturing the image provided below?

